Fraunhofer IPK
INSTITUT PRODUKTIONSANLAGEN UND KONSTRUKTIONSTECHNIK

BHT Berliner Hochschule für Technik

Sensor Integration into the Asset Administration Shell via OPC UA for a Milling Machine as a Foundation for Predictive Maintenance

Bachelor Thesis

Submitted by

Mazen Younis

Faculty Department VI – Computer Engineering
Berlin University of Applied Sciences and Technology

Submission Date: 25 January 2026

Supervisors

Prof. Dr.-Ing. Francisco Morales Serrano — Berliner Hochschule für Technik
M.Sc. Philipp Lelidis — Fraunhofer IPK Berlin

Studiere Zukunft

Abstract

Industrial Digital Twins rely on reliable acquisition, transport, and semantic exposure of machine and sensor signals. This thesis presents a modular data pipeline for an Exeron HSC600 milling machine that integrates (i) internal machine properties via OPC UA and (ii) external vibration and telemetry data from the Swarmy and ITENDO^2 sensor systems.

The objective is to establish a dependable, semantically aligned Digital Twin foundation using the Asset Administration Shell (AAS).

The solution is structured into three layers. First, a connection-resilient FastAPI middleware maintains a secure OPC UA session, periodically polls selected NodeIds, and exposes a schema-consistent HTTP JSON snapshot endpoint (`/snapshot`) providing values and timestamps. Second, a configuration-driven Eclipse BaSyx DataBridge pipeline (HTTP $\rightarrow$ JSONata $\rightarrow$ AAS) transfers these fields into Eclipse BaSyx submodels without application code changes, enabling semantically aligned and AAS-native access to machine state. Third, a unified frontend ingests external sensor streams via WebSockets, renders live visualizations with adaptive time windows, and persists all acquired time-series data in MongoDB. All components are containerized to ensure reproducible deployment.

In nominal runs, the UI renders at approximately 30 FPS with compact WebSocket payloads. For ITENDO^2, packet-loss indicators typically remained within the low single-digit percentage range, and representative OPC UA properties (e.g., `Chuck2IsClosed`) were verified as live values in the AAS viewer. While predictive-maintenance algorithms are out of scope, the architecture provides a reusable foundation for semantically addressable machine signals, near real-time monitoring, and future condition-monitoring and analytics applications.

Keywords: Asset Administration Shell, OPC UA, Eclipse BaSyx, FastAPI, JSONata, Digital Twin, time series, containerisation

Kurzfassung

Industrielle Digitale Zwillinge sind auf eine verlässliche Erfassung, Übertragung und semantische Bereitstellung von Maschinen- und Sensorsignalen angewiesen. Diese Arbeit stellt eine modulare Datenpipeline für die Fräsmaschine Exeron HSC600 vor, die (i) interne Maschinenzustände über OPC UA und (ii) externe Vibrations- und Telemetriedaten der Sensorsysteme Swarmy und ITENDO^2 integriert.

Ziel ist es, eine verlässliche und semantisch konsistente Grundlage für einen Digitalen Zwilling mithilfe der Asset Administration Shell (AAS) zu schaffen.

Die Lösung ist in drei Schichten gegliedert. Erstens hält eine wiederverbindungsfähige FastAPI-Middleware eine sichere OPC UA-Sitzung aufrecht, liest ausgewählte NodeIds periodisch aus und stellt einen schemakonsistenten HTTP-JSON-Schnappschuss (`/snapshot`) mit Zeitstempeln und Messwerten bereit. Zweitens überführt eine konfigurationsbasierte Eclipse BaSyx-DataBridge (HTTP $\rightarrow$ JSONata $\rightarrow$ AAS) diese Daten ohne Codeänderungen in Eclipse BaSyx-Submodelle und ermöglicht einen AAS-nativen Zugriff auf den Maschinenzustand. Drittens verarbeitet ein einheitliches Frontend externe Sensordatenströme über WebSockets, visualisiert diese nahezu in Echtzeit und persistiert alle Zeitreihen in MongoDB. Sämtliche Komponenten sind containerisiert ausgeführt, um ein reproduzierbares Deployment sicherzustellen.

Im Nominalbetrieb rendert die Benutzeroberfläche mit etwa 30 FPS bei kompakten WebSocket-Payloads. Für ITENDO^2 lagen die Paketverlustindikatoren typischerweise im niedrigen einstelligen Prozentbereich, und ausgewählte OPC UA-Eigenschaften (z. B. `Chuck2IsClosed`) wurden als Live-Werte im AAS-Viewer verifiziert. Obwohl prädiktive Instandhaltungsalgorithmen nicht Gegenstand dieser Arbeit sind, bietet die Architektur eine wiederverwendbare Grundlage für semantisch adressierbare Maschinensignale, zeitnahe Zustandsüberwachung und zukünftige Analyse- und Condition-Monitoring-Anwendungen.

Schlagwörter: Asset Administration Shell, OPC UA, Eclipse BaSyx, FastAPI, JSONata, Digitaler Zwilling, Zeitreihen, Containerisierung

Table of Contents

1 Introduction **1**

2 State of the Art **3**
2.1 Evolution of Digitalization in Manufacturing 3
2.2 Digital Twins in Manufacturing 4
2.3 Predictive Maintenance Approaches 5
2.4 Industrial Data Acquisition and Communication 6
2.5 Semantic Modeling of Machine Data 7
2.6 Middleware and Integration Architectures 8
2.7 Storage and Management of Time-Series Data 10
2.8 Discussion and Identified Research Gap 11

3 Problem Statement and Objectives **13**

4 System Environment and Technical Infrastructure **15**
4.1 System Environment and Experimental Context 15
4.2 Hardware and Software Environment 16
4.2.1 Hardware and infrastructure 16
4.2.2 Software and frameworks 17
4.3 References, Configuration, and Reproducibility 18
4.3.1 Licensing and third-party notices 19
4.3.2 Versioning note 19

5 System Design and Implementation **20**
5.1 Overall Architecture and Methodology 20
5.1.1 Architectural Principles and Methodological Approach 20
5.1.2 Implementation Process Stages 21
5.1.3 Goals, Scope, and Intended Outcome 23
5.2 Swarmy Sensor Integration 24
5.2.1 Swarmy Sensor System 24
5.2.2 Integration and Implementation 24
5.3 ITENDO^2 Sensor Integration 26
5.3.1 System Overview 26
5.3.2 Communication and Control 27
5.3.3 Backend Architecture 28
5.3.4 Frontend Visualization 28
5.3.5 Results and Observed Metrics 29
5.3.6 Configuration Notes 29
5.3.7 Control Flow and Session Coordination 29
5.4 OPC UA-Based Asset Administration Shell Integration (HSC600) 29
5.4.1 Architecture Overview 30
5.4.2 Middleware Implementation (FastAPI) 30

5.4.3 Signal Selection and Node Mapping 31
5.4.4 AAS Publication (Eclipse BaSyx DataBridge) 31
5.4.5 Containerization (Docker) 32
5.4.6 AAS Visualization in Eclipse BaSyx 32
5.4.7 Notes and Configuration 32

6 Evaluation and Testing **33**
6.1 Evaluation Objectives and Success Criteria 33
6.2 Testbed 34
6.3 Metrics and Instrumentation 34
6.4 Experiments 35
6.5 Results 36
6.6 Interpretation of Results 38
6.7 Threats to Validity 38

7 Discussion **40**

8 Conclusion and Outlook **43**
8.1 Conclusion 43
8.2 Outlook 43

9 References **45**

10 Appendix **48**

11 Acknowledgments **60**

List of Figures

Fig. 1 Overall system architecture of the implemented monitoring and integration pipeline. . . . 21
Fig. 2 Logical data flow and processing stages of the implemented monitoring and AAS integration pipeline. . . . 22
Fig. 3 Swarmy monitoring view with status, mode, battery, and packet-loss indicator. . . . 48
Fig. 4 Swarmy monitoring view with live three-axis acceleration time-series during active measurement. . . . 48
Fig. 5 Swarmy storage view with historical temperature, humidity, and battery voltage. . . . 49
Fig. 6 Swarmy transport view showing recorded High-G impact events. . . . 49
Fig. 7 ITENDO^2 monitoring view showing live acceleration data from the table-mounted sensor, including connection status and runtime indicators. . . . 50
Fig. 8 ITENDO^2 monitoring view showing live acceleration data from the spindle-mounted sensor during active measurement. . . . 50
Fig. 9 ITENDO^2 session-based data view with recorded measurement sessions, average packet-loss KPIs, and CSV export functionality. . . . 51
Fig. 10 Control-flow diagram for ITENDO^2 session handling: interaction between the user interface, backend services, ESP bridge nodes, and BLE-connected sensors. . . . 52
Fig. 11 ServerStatusSecondsTillShutdown property: remaining seconds before an automatic server shutdown. . . . 56
Fig. 12 OpenChuck2 command/state: open or closed status of chuck 2 as exposed via OPC UA and mapped into the AAS. . . . 56
Fig. 13 Chuck2IsClosed property: whether chuck 2 is currently closed, mapped from OPC UA to the AAS submodel. . . . 57
Fig. 14 Chuck2IsOccupied property: whether chuck 2 is currently occupied by a workpiece in automatic operation. . . . 57
Fig. 15 MaxHistoryContinuationPoints property: maximum number of continuation points maintained by the OPC UA server for historical reads. . . . 58
Fig. 16 MaxArrayLength property: maximum allowed length for array-type values in the OPC UA server. . . . 58
Fig. 17 SessionTimeoutCount property: number of sessions that have reached their configured timeout. . . . 59

List of Tables

Tab. 2 Hardware and environment: components used in the implementation and evaluation setup. 17
Tab. 3 Software and frameworks: versions used in the implementation and evaluation environment. 18
Tab. 4 Swarmy runtime services: REST and WebSocket endpoints. 25
Tab. 5 ITENDO^2 backend API: REST and WebSocket endpoints. 27
Tab. 6 OPC UA middleware HTTP API. 31
Tab. 7 Success criteria under nominal operation and corresponding target conditions. 33
Tab. 8 ITENDO^2 packet loss and reconnection: representative scenarios with per-sensor loss and recovery time (time to first valid UI update). 36
Tab. 9 Continuity metrics (representative sessions): expected versus missing samples and maximum gap duration derived from stored timestamps. . . 37
Tab. 10 AAS mapping verification: selected machine signals with NodeIds, target `idShortPath`, registry presence, and value consistency. 37
Tab. 11 Swarmy stream KPIs: packet-loss indicator, effective update rate, reconnection time, and gap characteristics under representative scenarios. . . 37
Tab. 12 Assessment of the results against the predefined success criteria (Table 7). 38

List of Listings

Listing 1 ITENDO^2 monitoring WebSocket payload (reduced stream): timestamp value pairs per sensor and Y-axis range hints. 28
Listing 2 Background polling loop in the OPC UA middleware: periodic NodeId reads, cache updates, and connection state tracking. 30
Listing 3 Continuity checker (pseudo-Python): estimate missing samples and gap durations from stored timestamps for a given session. 35
Listing 4 Recent window query (micro-benchmark): latency measurement for retrieving the last two minutes of samples. 35
Listing 5 Normalized response of the `GET /snapshot` endpoint: values and timestamps keyed by NodeId and friendly names. 53
Listing 6 Excerpt of the OPC UA NodeId mapping used by the middleware. . . 53
Listing 7 Robust OPC UA read with transparent reconnect on `BadSessionClosed` (client manager excerpt): single-retry logic after session re-creation. 53
Listing 8 DataBridge HTTP consumer: middleware `/snapshot` endpoint configuration for a selected signal. 54
Listing 9 JSONata extractor for `Chuck2IsClosed`: boolean value selection from the `values` object by NodeId. 54
Listing 10 AAS sink configuration (DotAAS V3): transformed value write to the specified `idShortPath`. 54
Listing 11 DataBridge timer definition: 1000 ms fixed-rate trigger for route execution. 54
Listing 12 DataBridge route example: HTTP consumer–JSONata–AAS sink connection bound to a timer trigger. 54
Listing 13 Docker Compose configuration (excerpt) for middleware and BaSyx DataBridge: port and volume mappings with host resolution for local development. 55
Listing 14 Dockerfile for the FastAPI middleware (Python 3.11-slim): dependency installation, source copying, and Uvicorn application exposure. 55

Symbols and Abbreviations

Abbreviation	Unit	Description
AAS	–	Asset Administration Shell
API	–	Application Programming Interface
ASGI	–	Asynchronous Server Gateway Interface
BLE	–	Bluetooth Low Energy
BSON	–	Binary JSON format used internally by MongoDB
CAN	–	Controller Area Network
CNC	–	Computer Numerical Control
CPS	–	Cyber-Physical System
CSV	–	Comma-Separated Values
DotAAS V3	–	REST-based API specification for AAS interaction
EWMA	–	Exponentially Weighted Moving Average
FPS	s^{-1}	Frames per second
GbE	$Gb\,s^{-1}$	Gigabit Ethernet
HSC600	–	Exeron HSC600 high-speed milling machine
HTTP	–	Hypertext Transfer Protocol
IDTA	–	Industrial Digital Twin Association
IIoT	–	Industrial Internet of Things
ISO 8601	–	Date and time representation standard
ITENDO^2	–	Wireless vibration and telemetry sensor system
JSON	–	JavaScript Object Notation
JSONata	–	Query and transformation language for JSON
KPI	–	Key Performance Indicator
LSB	–	Least Significant Bit
Modbus	–	Industrial serial communication protocol
MongoDB	–	NoSQL document and time-series database
MQTT	–	Publish–subscribe messaging protocol
NC	–	Numerical Control
NodeId	–	Unique identifier in the OPC UA address space
npm	–	Node Package Manager
OPC UA	–	Open Platform Communications Unified Architecture
PLC	–	Programmable Logic Controller
PROFIBUS	–	Industrial fieldbus communication protocol
REST	–	Representational State Transfer
RFC 6455	–	WebSocket protocol specification
RTT	s	Round-trip time
TCP	–	Transmission Control Protocol
TSDB	–	Time-Series Database

Abbreviation	Unit	Description
UART	–	Universal Asynchronous Receiver–Transmitter
UI	–	User Interface
UTC	–	Coordinated Universal Time
WebSocket	–	Full-duplex communication protocol over TCP

1 Introduction

Modern manufacturing within the scope of Industry 4.0 increasingly relies on the systematic acquisition and use of digital machine data. Production systems are no longer composed solely of isolated machines, but of interconnected assets that generate heterogeneous data streams originating from machine controllers, embedded sensors, and external measurement devices. To exploit this data for monitoring, optimization, and advanced analytics, it must be integrated into unified, interpretable, and machine-readable representations [KLE17, AKI18]. A central challenge, however, lies not merely in acquiring data, but in ensuring that information remains semantically consistent, traceable, and interoperable across systems, vendors, and lifecycle phases [PLA23, AAS20]. Without a shared semantic foundation, data integration efforts tend to become fragile, difficult to scale, and prone to misinterpretation.

A key concept addressing these challenges is the **Digital Twin**. A Digital Twin represents a dynamic digital counterpart of a physical asset that reflects its structure, state, and behavior based on operational data [TAO19]. By linking the physical and digital domains through continuous data exchange, Digital Twins enable applications such as monitoring, diagnostics, and process optimization. For such applications to remain robust and reusable over time, Digital Twins depend on standardized data models that clearly define asset semantics and access mechanisms along the value chain [PLA23, AAS20].

Within the European Industry 4.0 initiative, this standardization effort is realized through the **Asset Administration Shell (AAS)** [PLA23]. The AAS provides a structured and standardized digital representation of an asset and is regarded as the official realization of the Digital Twin concept in this context [AAS20]. It encapsulates asset identity and organizes information into modular submodels that describe properties, capabilities, and operational data. This modular organization supports interoperability across organizational and technological boundaries while preserving semantic clarity and extensibility [IDT24].

Complementing standardized information models, reliable industrial communication is essential for accessing machine data in a consistent manner. **Open Platform Communications Unified Architecture (OPC UA)** has emerged as a widely adopted standard for secure and structured machine communication [OPC24, MAH09]. OPC UA defines both a communication protocol and an information model that enable reliable access to machine states, process variables, and diagnostic data. Modern machine tools commonly expose such information through integrated OPC UA servers, making the standard a key enabler for vertical data integration within Industry 4.0 environments.

The relevance of these technologies becomes particularly evident in the context of predictive maintenance. Predictive maintenance aims to anticipate machine failures and performance degradation by analyzing operational data over time, allowing maintenance actions to be planned before faults occur [JAR06, TAO19]. Such approaches rely not only on the availability of data, but also on its semantic consistency, traceability, and long-term interpretability across systems and lifecycle phases [LEE14, KLE17]. In industrial environments, however, heterogeneous data sources, proprietary interfaces, and inconsistent signal semantics often hinder the

systematic use of machine data for predictive maintenance applications [LEE14, PLA23]. As a result, significant effort is frequently spent on data integration and interpretation rather than on analytical modeling itself [KLE17, AKI18].

This thesis addresses these challenges by establishing a structured, standards-based foundation for machine data integration using OPC UA, the Asset Administration Shell, and the open-source framework **Eclipse BaSyx** [BAS25]. Eclipse BaSyx provides a runtime environment for hosting and managing AAS-based digital representations and enables the integration of external data sources into semantically structured AAS submodels. Rather than implementing predictive maintenance algorithms directly, the focus of this work lies on reliable data acquisition, semantic alignment, and traceability between physical machine signals and their corresponding Digital Twin representations. In this sense, the presented solution serves as an enabling foundation for future predictive maintenance applications rather than as a complete predictive maintenance system [PLA23, AAS20].

The work was conducted at Fraunhofer IPK within the context of the EMOTION research project [EMO25], which focuses on edge-based monitoring and optimization for industrial machining processes. The experimental setup comprises an Exeron HSC600 high-speed milling machine equipped with an integrated OPC UA server, as well as external sensors (Swarmy and ITENDO^2) for additional process monitoring [OPC24, MAH09]. Within this project context, selected machine signals are semantically mapped into an Asset Administration Shell and made accessible through Eclipse BaSyx, while external sensor data are integrated at the visualization and storage layers. The resulting system enables unified live monitoring and provides a consistent, semantically structured digital representation of selected machine states, forming a reproducible basis for future data-driven applications in industrial environments [BAS25, AAS20, PLA23].

2 State of the Art

2.1 Evolution of Digitalization in Manufacturing

The digitalization of manufacturing has progressed in several waves, driven by advances in automation technology, computing power, and industrial networking. Early production systems were dominated by mechanically controlled processes and isolated machine operations. With the introduction of programmable logic controllers (PLCs), numerical control, and later computer-integrated manufacturing, automation became increasingly software-driven and enabled higher productivity through repeatable, parameterized process control [LEE14]. Over time, manufacturers also began to connect production equipment to supervisory control systems and enterprise IT, motivated by the need for transparency, quality assurance, and more responsive planning.

A key characteristic of the pre-Industry 4.0 era, often associated with Industry 3.0, is that digitalization primarily focused on automation within individual machines or production cells, while end-to-end integration across the shop floor and higher-level systems remained limited. Data acquisition was frequently constrained by proprietary machine interfaces and vendor-specific protocols, resulting in isolated “data silos” that were costly to integrate and difficult to maintain over the lifecycle of equipment [LEE14]. These limitations became particularly apparent as sensor-based monitoring and condition-based maintenance gained relevance, because reliable diagnostics require consistent access to operational signals over long time horizons [JAR06]. In practice, the effort required to collect and interpret heterogeneous signals often outweighed the benefits of analysis, slowing down the adoption of data-driven methods [KLE17].

The concept of Industry 4.0 builds upon these developments by promoting vertical and horizontal integration, where machines, sensors, and software services exchange data across organizational and system boundaries [PLA23]. This shift is closely linked to the notion of Cyber-Physical Systems (CPS), which connect physical processes to computational control and networked communication, enabling distributed intelligence and adaptive behavior in production environments [LEE14]. From a data perspective, CPS-oriented manufacturing increases the importance of robust transport mechanisms and scalable data pipelines, because operational information must be acquired, processed, and made available for monitoring and decision support with low latency and high reliability [AKI18]. At the same time, the broader system scope increases the complexity of managing data semantics and ensuring that signals remain interpretable across changing software versions, organizational interfaces, and lifecycle phases [KLE17].

As manufacturing systems become more connected, the focus of digitalization shifts from isolated automation toward integrated, data-driven operations. This development provides the technological foundation for advanced applications such as predictive maintenance, where failures and degradation are anticipated based on continuously collected machine and sensor data [WANG21]. However, the evolution toward Industry 4.0 also exposes persistent challenges: heterogeneous interfaces, inconsistent signal semantics, and fragmented integration

practices continue to hinder the systematic reuse of production data for analytics and optimization [LEE14]. These challenges motivate the emergence of standardized communication and modeling approaches, which are discussed in the following sections.

2.2 Digital Twins in Manufacturing

The concept of the Digital Twin originated from early ideas of mirroring physical systems with digital representations to support design, simulation, and lifecycle management. One of the most influential early formulations was introduced by Michael Grieves, a researcher in product lifecycle management, who proposed the notion of a digital counterpart that accompanies a physical product throughout its lifecycle to enable feedback-driven improvement and decision support [GRI17]. In this view, the Digital Twin is not merely a static model, but a living digital artifact that evolves together with its physical counterpart as new data become available.

Since its introduction, the term Digital Twin has been adopted and interpreted in various ways across research communities and industrial domains. In manufacturing-oriented literature, Digital Twins are commonly defined as dynamic digital representations of physical assets that integrate real-world data to reflect structure, state, and behavior over time [TAO19]. Other definitions emphasize the role of continuous data exchange and synchronization between the physical and digital domains, highlighting that a Digital Twin must be connected to its physical counterpart through bidirectional data flows rather than relying on offline models alone [KRA23]. These differing perspectives reveal that the concept spans a spectrum ranging from simulation-driven digital models to fully integrated, data-driven system representations.

A recurring distinction in the literature concerns the level of fidelity and coupling between the physical asset and its digital representation. While early digital models were often limited to design-time simulations or engineering documentation, contemporary Digital Twin approaches increasingly incorporate operational data streams from sensors and control systems [TAO19]. This shift enables the Digital Twin to support runtime use cases such as monitoring, diagnostics, and optimization, rather than serving purely analytical or planning purposes. At the same time, the lack of a universally accepted definition has led to ambiguity in how Digital Twins are implemented and evaluated, making it difficult to compare solutions across vendors and research projects [KRA23].

Recent research trends reflect a growing interest in standardization and interoperability to address these challenges. In manufacturing contexts, Digital Twins are increasingly discussed in relation to Cyber-Physical Systems and Industry 4.0 architectures, where they act as semantic integration points between machines, software services, and analytics applications [LEE14]. Surveys indicate a strong focus on enabling cross-system data exchange, lifecycle consistency, and scalable deployment of Digital Twins across production environments [KRA23]. Nevertheless, many implementations remain tightly coupled to specific platforms or data sources, limiting reuse and long-term maintainability.

Overall, the evolution of the Digital Twin concept in manufacturing highlights both its potential and its current limitations. While the idea of a continuously updated digital representation has

become widely accepted, practical realization varies significantly in terms of scope, semantics, and integration depth. These observations underline the need for standardized models and interfaces that can anchor Digital Twins within heterogeneous industrial environments, a topic that is further explored in the subsequent sections.

2.3 Predictive Maintenance Approaches

Maintenance strategies in industrial production have traditionally evolved in response to increasing system complexity, cost pressure, and availability requirements. Early manufacturing environments primarily relied on reactive maintenance, where corrective actions were performed only after a failure occurred. While this approach minimizes upfront effort, it often leads to unplanned downtime, secondary damage, and high overall maintenance costs, making it unsuitable for modern, highly automated production systems [JAR06].

To mitigate these risks, preventive maintenance strategies were introduced, in which maintenance actions are scheduled based on fixed intervals or usage metrics. Preventive approaches improve equipment reliability compared to purely reactive strategies, but they do not account for the actual condition of machine components. As a result, maintenance may be performed either too early, wasting resources, or too late, failing to prevent unexpected failures [WANG21]. These limitations motivated the development of more data-driven approaches that adapt maintenance decisions to the observed behavior of machines.

Predictive maintenance represents a further evolution by aiming to anticipate failures and performance degradation before they occur, based on continuous monitoring and analysis of operational data [JAR06]. Rather than relying on predefined schedules, predictive maintenance leverages machine signals, sensor measurements, and historical data to estimate the remaining useful life of components and to detect anomalous behavior indicative of emerging faults. In industrial contexts, predictive maintenance is widely regarded as a key enabler for reducing downtime, optimizing maintenance planning, and improving overall equipment effectiveness [SIEM18, WANG21].

A central class of predictive maintenance methods is condition-based maintenance, where decisions are derived from the current and evolving condition of an asset. Condition indicators such as vibration, temperature, force, or power consumption are continuously evaluated to assess machine health [JAR06]. Traditional condition-based approaches often rely on threshold-based rules or expert-defined features, while more recent data-driven methods apply statistical models and machine learning techniques to capture complex patterns in multivariate sensor data [WANG21]. These approaches benefit from increasing sensor availability and computational capabilities but also introduce new requirements for data quality, consistency, and interpretability.

Despite significant research progress, the practical implementation of predictive maintenance in industrial environments remains challenging. One major obstacle is the heterogeneous nature of data sources, which include machine controllers, external sensors, and legacy systems using proprietary interfaces [LEE14]. Integrating these sources into a unified data pipeline often

requires substantial engineering effort, particularly when signal semantics are undocumented or inconsistent across vendors and machine generations. As a consequence, a large portion of industrial predictive maintenance projects focuses on data acquisition and preprocessing rather than on analytical modeling itself [KLE17].

Another challenge concerns the long-term availability and traceability of data across system and lifecycle boundaries. Predictive maintenance models typically require historical data collected over extended periods, yet changes in software versions, machine configurations, or data schemas can compromise comparability and reuse [LEE14]. Furthermore, the lack of standardized semantic models for machine and sensor data makes it difficult to relate analytical results back to specific physical components or operational contexts, limiting transparency and trust in predictive maintenance solutions [PLA23].

Overall, the state of the art demonstrates that predictive maintenance has matured conceptually and methodologically, but its industrial adoption is still constrained by integration complexity, semantic inconsistency, and fragmented toolchains. These limitations highlight the need for standardized data access mechanisms and semantically structured digital representations that can serve as a reliable foundation for predictive maintenance applications across heterogeneous manufacturing environments.

2.4 Industrial Data Acquisition and Communication

The acquisition and transmission of machine and sensor data have long been fundamental challenges in industrial automation. Historically, manufacturing systems relied on proprietary machine interfaces and vendor-specific communication protocols, which were tightly coupled to individual control platforms and hardware ecosystems. While these solutions enabled reliable real-time control within isolated production cells, they significantly limited interoperability and made cross-vendor integration complex and costly over the lifecycle of industrial equipment [LEE14, MAH09].

To address real-time communication requirements on the shop floor, a wide range of fieldbus systems emerged, including PROFIBUS, CAN, Modbus, and later Industrial Ethernet variants. These technologies were optimized for deterministic communication, low latency, and robustness in harsh industrial environments. However, their primary focus remained on control-level data exchange rather than semantic interoperability or integration with higher-level IT systems [MAH09]. As a result, data extracted from field-level communication often required extensive preprocessing and contextualization before it could be used for monitoring, diagnostics, or analytics.

With the increasing demand for transparency and data-driven optimization, industrial communication evolved beyond field-level connectivity toward standardized, service-oriented architectures. A major milestone in this evolution is Open Platform Communications Unified Architecture (OPC UA), which combines a platform-independent communication protocol with a rich information modeling framework [OPC24]. Unlike traditional fieldbus systems, OPC UA enables structured access to machine data, metadata, and relationships through a unified address space,

supporting both synchronous requests and asynchronous subscriptions [MAH09]. This design makes OPC UA particularly suitable for vertical integration scenarios, where data must flow reliably from the shop floor to supervisory systems and enterprise applications.

In addition to its technical features, OPC UA addresses key non-functional requirements such as security, scalability, and vendor neutrality. Built-in mechanisms for authentication, authorization, and encryption allow secure data exchange across network boundaries, while standardized information models promote consistent interpretation of machine signals across systems and organizational units [OPC24]. Consequently, OPC UA has become a de facto standard for industrial data access in Industry 4.0 environments and is widely supported by modern machine tools and automation platforms.

Alongside OPC UA, lightweight messaging protocols such as Message Queuing Telemetry Transport (MQTT) have gained popularity, particularly in Industrial Internet of Things (IIoT) scenarios. MQTT follows a publish subscribe paradigm optimized for low-bandwidth and unreliable networks, making it well suited for large-scale sensor deployments and cloud-based data ingestion pipelines [HONG18]. Compared to OPC UA, MQTT offers lower communication overhead and simpler client implementations but does not define a standardized information model or data semantics.

Comparative studies highlight that OPC UA and MQTT address complementary requirements rather than serving as direct substitutes [HONG18]. OPC UA excels in structured, semantically rich access to machine data and tight integration with industrial control systems, whereas MQTT prioritizes scalability and efficiency for event-based data distribution. In practice, industrial architectures often combine both approaches, using OPC UA for machine-level data access and MQTT or similar protocols for downstream data dissemination. Nevertheless, this coexistence further emphasizes the importance of semantic consistency and standardized modeling to ensure that data remains interpretable across heterogeneous communication layers [LEE14].

Overall, the state of the art in industrial data acquisition reflects a transition from isolated, proprietary communication toward standardized and service-oriented interfaces. While modern protocols such as OPC UA significantly improve accessibility and interoperability, the coexistence of multiple communication technologies continues to pose challenges for unified data integration, traceability, and long-term maintainability in data-driven manufacturing systems.

2.5 Semantic Modeling of Machine Data

As manufacturing systems become increasingly interconnected, the availability of raw machine data alone is no longer sufficient to support monitoring, diagnostics, and optimization tasks. Industrial data typically originate from heterogeneous sources, differ in structure and meaning, and evolve over the lifecycle of machines and software systems. Without explicit semantic descriptions, the interpretation of signals often depends on implicit knowledge held by engineers or encoded in proprietary interfaces, which hinders scalability, reuse, and long-term maintainability [LEE14, KLE17].

Semantic modeling addresses this challenge by enriching data with machine-readable meaning that defines what a signal represents, how it relates to other data, and how it can be used in a broader system context. In industrial environments, semantics are essential for ensuring that data remain interpretable across organizational boundaries, system updates, and changing usage scenarios [PLA23]. This is particularly relevant for data-driven applications such as predictive maintenance, where consistent interpretation of time-series signals over long periods is required to detect trends, anomalies, and degradation patterns reliably [JAR06].

Within the context of Industry 4.0, the Asset Administration Shell (AAS) has been introduced as a standardized approach to semantic modeling of industrial assets [PLA23]. The AAS provides a structured digital representation that encapsulates all relevant information about an asset, including identification, technical properties, operational data, and functional capabilities. This information is organized into modular submodels, each addressing a specific aspect such as nameplate data, condition monitoring, or maintenance information [AAS20]. By separating concerns into submodels, the AAS supports extensibility while preserving a common structural and semantic foundation.

A central design principle of the AAS is the explicit use of semantic identifiers that reference globally defined concepts, enabling unambiguous interpretation of data elements across systems and organizations [IDT24]. In contrast to ad hoc data integration approaches, this model-based representation allows consumers to discover, interpret, and reuse asset information without prior knowledge of vendor-specific data structures. As a result, the AAS is often positioned as the standardized realization of the Digital Twin concept within the European Industry 4.0 initiative [AAS20].

Despite its conceptual strengths, the adoption of semantic modeling approaches such as the AAS remains uneven in practice. Surveys and case studies indicate growing interest in research and pilot projects, particularly in areas related to interoperability, lifecycle management, and cross-company data exchange [KRA23, CAV20]. However, widespread industrial adoption is challenged by factors such as implementation complexity, the need for domain-specific modeling expertise, and the coexistence of legacy systems that lack semantic descriptions [PLA23]. Furthermore, the integration of semantic models with existing data pipelines and industrial communication infrastructures continues to require significant engineering effort.

Overall, the state of the art demonstrates a clear recognition of the importance of semantic modeling for data-driven manufacturing systems. While standardized concepts such as the Asset Administration Shell provide a promising foundation, practical realization and consistent usage across heterogeneous industrial environments remain open challenges. These limitations motivate continued research into scalable integration architectures and supporting middleware, which are discussed in the following sections.

2.6 Middleware and Integration Architectures

The increasing connectivity of manufacturing systems has led to a growing number of heterogeneous data sources that must be integrated into coherent digital infrastructures. Machines,

sensors, enterprise systems, and cloud services often rely on different communication protocols, data formats, and update rates, making direct point-to-point integration difficult to scale and maintain. To address this complexity, middleware has emerged as a key architectural layer in Industrial Internet of Things (IIoT) and Industry 4.0 systems, providing abstraction, decoupling, and coordination between data producers and consumers [VERM18, DAZA20].

IIoT middleware platforms typically offer functionalities such as protocol translation, data normalization, buffering, and routing, enabling applications to access industrial data without direct knowledge of underlying device interfaces [DAZA20]. By separating data acquisition from data consumption, middleware supports modular system design and facilitates the integration of legacy equipment alongside modern, network-enabled devices. However, surveys indicate that middleware solutions vary widely in scope and design philosophy, ranging from lightweight message brokers to comprehensive platforms that include device management, analytics, and visualization capabilities [VERM18].

In parallel, the rise of data-intensive applications has driven the adoption of streaming and pipeline-based architectures for industrial data processing. Instead of treating data as isolated requests, these architectures model data flows as continuous streams that are ingested, processed, and stored in near real time [AKI18]. Foundational work on data-intensive systems highlights the benefits of such approaches in terms of scalability, fault tolerance, and temporal consistency, particularly when dealing with high-frequency sensor data and long-running analytical tasks [KLE17]. Within manufacturing contexts, streaming pipelines enable low-latency monitoring and provide a foundation for advanced analytics such as anomaly detection and predictive maintenance.

Despite these advances, integrating middleware with semantic models and industrial communication standards remains a significant challenge. Many middleware platforms focus primarily on data transport and transformation, leaving semantic interpretation to application-specific logic [KLE17]. As a result, data may be syntactically unified while remaining semantically ambiguous, limiting reuse across applications and organizational boundaries. Furthermore, the coexistence of event-driven streaming mechanisms and request-based service interfaces introduces additional architectural complexity, particularly when consistency and traceability across system layers are required [AKI18].

Another challenge concerns the long-term maintainability of integration architectures in industrial environments. Manufacturing systems are characterized by long equipment lifecycles, evolving software components, and changing organizational requirements. Middleware solutions must therefore balance flexibility with stability, ensuring that integration logic can adapt without disrupting ongoing operations [VERM18]. Studies emphasize that the lack of standardized integration patterns and the fragmentation of available tools continue to hinder systematic reuse and large-scale deployment of IIoT solutions [DAZA20].

In summary, middleware and integration architectures play a central role in enabling data-driven manufacturing by decoupling heterogeneous systems and supporting scalable data flows. While streaming pipelines and IIoT middleware platforms provide powerful mechanisms for data transport and processing, their effective combination with semantic models and in-

dustrial standards remains an open challenge. Addressing this gap is essential for achieving end-to-end consistency and traceability in complex manufacturing systems, and motivates the architectural approaches discussed in later chapters.

2.7 Storage and Management of Time-Series Data

Industrial manufacturing systems generate large volumes of time-stamped data originating from machines, sensors, and control systems. Typical examples include process variables, condition-monitoring signals, event logs, and diagnostic indicators. Such data are characterized by high write rates, temporal ordering, and a strong dependence on time for interpretation, as values often derive meaning from trends, correlations, and temporal context rather than from individual samples alone [KLE17]. In addition, industrial time-series data frequently exhibit irregular sampling intervals, missing values, and varying signal quality, reflecting the realities of operational environments and long equipment lifecycles [JEN17].

Traditional relational database management systems (RDBMS) were originally designed for transactional workloads and structured business data. While they can store time-stamped records, their performance and scalability often degrade when faced with continuous high-frequency inserts and large historical datasets typical of industrial monitoring scenarios [KLE17]. As a result, specialized time-series data management systems have emerged, offering data models and storage mechanisms optimized for append-only workloads, efficient time-based queries, and compression of temporally correlated values [JEN17].

Time-series databases (TSDBs) typically organize data around a timestamp as the primary dimension, often combined with lightweight metadata describing the signal source, such as a sensor identifier or machine context. Survey literature highlights features such as automatic data partitioning, downsampling, and retention policies as essential for managing the long-term growth of industrial datasets while maintaining query performance [JEN17]. These capabilities support both short-term monitoring use cases, which require fast access to recent data, and long-term analysis tasks that operate on historical trends.

From an application perspective, the suitability of a storage system is closely linked to the intended analytical workload. Monitoring dashboards and alarm systems demand low-latency access to recent values and efficient range queries, whereas diagnostic and predictive analytics often require aggregation, resampling, and correlation across multiple signals over extended time periods [AKI18]. Consequently, time-series storage is frequently integrated into broader data pipelines that combine streaming ingestion with batch-oriented analysis, allowing systems to balance responsiveness and analytical depth [KLE17].

Despite these advances, challenges remain in aligning time-series storage with semantic models and higher-level representations of industrial assets. Time-series databases typically focus on efficient storage and retrieval of numerical values, leaving semantic interpretation, such as the meaning of a signal, its unit, or its relation to a physical component, to external systems [PLA23]. This separation can complicate traceability and reuse, particularly when data must be interpreted consistently across different applications or organizational contexts.

In summary, effective storage and management of time-series data are a foundational requirement for data-driven manufacturing applications. Specialized time-series databases address many performance and scalability limitations of traditional relational systems and are well suited for monitoring and analytics workloads. However, their integration into semantically rich industrial architectures remains an open issue, underscoring the need for coordinated approaches that combine efficient data storage with standardized asset models and interoperable interfaces.

2.8 Discussion and Identified Research Gap

The reviewed literature demonstrates significant progress in the digitalization of manufacturing systems over the past decades. Advances in automation, industrial communication, and data management have enabled increasingly data-driven operation of production assets. Concepts such as Cyber-Physical Systems and Digital Twins provide a conceptual foundation for integrating physical processes with digital representations, while predictive maintenance approaches illustrate the potential value of continuous condition monitoring and analytics for industrial operations [LEE14, TAO19, WANG21].

At the same time, the analysis of existing approaches reveals several persistent limitations. Despite widespread availability of sensor data and standardized communication technologies such as OPC UA, industrial data integration remains complex and fragmented. Many solutions rely on proprietary interfaces, project-specific data models, or tightly coupled middleware stacks, which hampers interoperability and long-term maintainability [MAH09, HONG18]. Even where standardized communication is used, the semantic meaning of machine signals is often insufficiently specified, leading to ambiguity when data are reused across systems, applications, or lifecycle phases [PLA23].

Digital Twin implementations further highlight this challenge. While the concept is widely adopted in research and industry, practical realizations vary considerably in scope, coupling depth, and semantic rigor [KRA23]. Many Digital Twins remain limited to visualization or simulation use cases and lack a standardized, machine-interpretable structure that would support scalable integration, traceability, and reuse across heterogeneous environments [GRI17]. The absence of a common semantic backbone complicates the consistent interpretation of operational data and restricts the applicability of Digital Twins as integration artifacts within larger industrial architectures.

The Asset Administration Shell addresses several of these shortcomings by providing a standardized model for representing industrial assets and their properties in a semantically explicit manner [AAS20]. However, existing literature and reported implementations indicate that the integration of live machine data into AAS-based structures is still at an early stage, particularly with respect to automated data mapping, runtime operation, and practical deployment in mixed OT/IT environments [CAV20]. Moreover, many implementations focus either on modeling or on communication, without fully addressing end-to-end traceability from physical signals to their semantic representations.

From a data management perspective, time-series databases and streaming architectures offer efficient mechanisms for storing and processing industrial data at scale [KLE17, AKI18]. Nevertheless, these technologies typically operate at a syntactic level and do not inherently preserve semantic context, requiring additional layers to relate stored values to asset structures, engineering meaning, or functional roles [JEN17]. As a result, integration between data storage systems and semantic asset models remains largely manual or application-specific.

In summary, the current state of the art provides powerful individual technologies for communication, data storage, and digital representation, yet lacks a cohesive, standards-based integration approach that combines these elements into a consistent and traceable architecture. The identified research gap lies in the systematic integration of live industrial machine data into standardized Digital Twin representations, ensuring semantic consistency, interoperability, and traceability across system boundaries. Addressing this gap is a prerequisite for scalable data-driven applications such as predictive maintenance and forms the motivation for the approach presented in the subsequent chapters.

3 Problem Statement and Objectives

The state of the art shows that machine connectivity, Digital Twin concepts, and data-driven maintenance are well established in principle, yet end-to-end integration in real manufacturing environments remains difficult due to heterogeneous data sources, long equipment lifecycles, and evolving software and data schemas [LEE14, KLE17, AKI18]. In particular, modern Computer Numerical Control (CNC) machine tools provide increasing access to internal controller data, while additional process- and condition-relevant information is often available only through separate external sensing. This creates a practical gap between data availability and dependable, interpretable integration across machine and sensor systems.

This thesis investigates these challenges in an industrial machining testbed within the context of the EMOTION research project [EMO25]. The test environment consists of an Exeron HSC600 high-speed milling machine with access to an integrated Open Platform Communications Unified Architecture (OPC UA) server for structured machine-side signals [OPC24, MAH09]. In addition, external sensors provide complementary measurements: Swarmy (serial interface) and the iTENDO^2 system (network streams via Transmission Control Protocol, TCP) [SCH25a, SCH25b]. While OPC UA improves standardized access to machine data, it does not automatically resolve the integration of external streams nor guarantee semantic consistency across all sources [OPC24]. At the semantic level, the Asset Administration Shell (AAS) provides a standardized Digital Twin representation for industrial assets, intended to enable interoperable and machine-interpretable modeling via modular submodels and explicit semantic references [AAS20, PLA23, IDT24]. However, consistently mapping live signals into such semantic structures and operating a unified pipeline under realistic runtime conditions remains a non-trivial engineering task [KRA23].

Problem statement. Even in a laboratory setting with modern standards, the joint use of (i) machine-internal OPC UA signals and (ii) external sensor streams leads to integration effort and ambiguity: differences in protocols, sampling behavior, payload formats, and implicit semantics hinder consistent interpretation, traceability, and reuse across components [KLE17, AKI18, PLA23]. In addition, reliable operation requires robustness against disconnects, bursty transmissions, and non-ideal communication behavior that must remain observable for correct data interpretation and later analytics [AKI18, KLE17].

Main goal of the thesis. The main goal of this thesis is to design and implement a dependable, standards-oriented data pipeline for the Exeron HSC600 that integrates heterogeneous data streams for live monitoring and persistent storage, while mapping a defined subset of machine-side signals into a semantically structured Digital Twin representation based on the Asset Administration Shell [AAS20, PLA23, BAS25].

The main goal is divided into the following sub-goals:

1. **Sub-goal 1: Integration of machine-side OPC UA signals into the AAS.**
   Acquire a selected subset of Exeron HSC600 signals exposed via OPC UA and map

them into AAS submodels hosted in an AAS runtime environment. The mapped properties shall be addressable and interpretable via AAS structures (e.g., consistent identifiers and semantically aligned elements), using a deployment compatible with Eclipse BaSyx [OPC24, AAS20, BAS25].

2. **Sub-goal 2: Unified acquisition, visualization, and storage of external sensor streams.** Implement reliable ingestion for Swarmy (serial) and iTENDO^2 (TCP) data streams, forward live updates to a single web-based user interface via a WebSocket endpoint, and persist all samples as time-series data for later inspection and analysis [RFC11, MON21, KLE17]. This integration is performed while keeping external sensors outside formal AAS modeling in this phase, i.e., integrated at UI and database layers with contextual association to the machine [PLA23, IDT24].

3. **Sub-goal 3: Runtime robustness and observability under realistic conditions.** Ensure stable runtime behavior under intermittent connectivity and variable data rates by implementing reconnection handling, continuity tracking, and observable runtime indicators (e.g., connection status and packet loss measures for iTENDO^2). These indicators shall support transparent assessment of system behavior and improve interpretability of recorded datasets [AKI18, KLE17].

Constraints and scope. The work is constrained to a reproducible, containerized deployment of all components [MER14]. OPC UA-to-AAS mappings shall be configuration-driven (declarative) to simplify extension and maintenance, leveraging existing transformation mechanisms where applicable [BAS25, JSO25]. Semantic modeling is limited to a defined subset of machine-side signals; full semantic embedding of external sensors into the AAS is explicitly excluded in this implementation phase [AAS20, IDT24]. Furthermore, this thesis does not implement predictive-maintenance algorithms or machine-learning models, and does not conduct exhaustive throughput/latency benchmarking; the focus lies on dependable acquisition, integration, and interpretable runtime behavior [TAO19, AKI18].

Success criteria. The implementation is considered successful if (i) live OPC UA values are correctly exposed through the AAS viewer with the intended submodel structure, (ii) Swarmy and iTENDO^2 streams are acquired continuously, visualized live, and stored persistently in MongoDB time-series collections, and (iii) runtime indicators (including connectivity status and representative packet loss observations for iTENDO^2) are available to validate dependable operation during representative sessions [OPC24, BAS25, RFC11, MON21].

4 System Environment and Technical Infrastructure

This chapter describes the system environment and experimental setup in which the work presented in this thesis was conducted. The focus lies on the physical testbed, the involved hardware and software components, and the organizational and technical context that defines the boundary conditions for design, implementation, and evaluation. Together, these elements constitute the initial conditions under which the developed data integration architecture operates and ensure reproducibility of the reported results.

4.1 System Environment and Experimental Context

Research and laboratory environment. The work was carried out at the Fraunhofer Institute for Production Systems and Design Technology (IPK) in Berlin within the context of the EMOTION research project [IPK25, EMO25]. Fraunhofer IPK conducts applied research in the fields of production systems, digital manufacturing, and data-driven optimization. The experimental setup used in this thesis is located in a controlled laboratory environment and is intended for research and development purposes rather than continuous production operation. Consequently, the system reflects realistic industrial conditions with respect to machine interfaces, data rates, and communication behavior, while remaining accessible for experimentation and iterative development.

Machining system. The central asset of the experimental setup is an Exeron HSC600 high-speed milling machine. As a modern computer numerical control (CNC) machining center, the machine performs automated material removal operations based on numerically programmed tool paths. The machine is equipped with an integrated control system that exposes internal machine states, process variables, and diagnostic information via an OPC UA server interface [MAH09, OPC24]. This interface provides structured and standardized access to selected machine signals and serves as the primary source of machine-internal data within the scope of this work.

External sensor systems. In addition to the machine-internal data provided by OPC UA, the experimental setup includes external sensors that capture supplementary process- and condition-related information. Two sensor systems are considered in this thesis. The first is Swarmy, which communicates via a serial interface and delivers time-series data streams related to the machining process. The second is the ITENDO^2 tool holder system, which provides measurement data through network-based communication using Transmission Control Protocol (TCP) sockets [SCH25a, SCH25b]. These sensors operate independently of the machine control and illustrate a common industrial scenario in which relevant information is distributed across heterogeneous devices and interfaces.

Software and communication infrastructure. Machine-internal data are accessed through the OPC UA interface of the Exeron HSC600, which supports structured data models and subscription-based updates [OPC24]. External sensor data are acquired through dedicated backend components that handle serial and socket-based communication. For downstream processing and visualization, all acquired data streams are forwarded to a unified software infrastructure that supports live monitoring and persistent storage of time-series data [RFC11, MON21]. The system architecture is designed to operate within containerized environments to ensure reproducibility and consistent deployment across development and test systems [MER14].

Semantic integration context. Within the experimental setup, semantic modeling is applied selectively. Machine-internal signals accessed via OPC UA are mapped into Asset Administration Shell (AAS) submodels, providing a standardized and semantically structured digital representation of the machine [AAS20, PLA23]. External sensor data are intentionally excluded from direct AAS modeling and are instead integrated at the visualization and data storage layers. This separation reflects current industrial practice and allows investigation of how heterogeneous data sources can be contextually associated with a machine-centric Digital Twin without enforcing full semantic integration for all sensors [IDT24, TAO19].

Tools, frameworks, and runtime environment. Beyond the physical setup, the implementation and evaluation of the system rely on a defined set of software tools, middleware components, and runtime environments. These tools cover industrial connectivity (OPC UA), Digital Twin infrastructure based on the Asset Administration Shell, time-series data persistence, and web-based services for backend processing and frontend visualization [OPC24, MAH09, PLA23, BAS25, MON21, FAS25, QUA25, REA25, CHA25]. Documenting these components provides a reproducible reference for the technical environment that influences the reported results.

4.2 Hardware and Software Environment

4.2.1 Hardware and infrastructure

Table 2 summarizes the hardware and connectivity components used in the implementation and evaluation setup. It lists the Exeron HSC600 milling machine as the primary asset, the external sensor systems (Swarmy and iTENDO^2), and the edge-host infrastructure responsible for data acquisition, processing, storage, and visualization. This overview defines the technical boundary conditions of the testbed and provides the context required to interpret the integration design and the evaluation results presented in later chapters.

Tab. 2: Hardware and environment: components used in the implementation and evaluation setup.

Component	**Version / Model**	**Role / Notes**
Exeron HSC600	Heidenhain TNC 640; OPC UA NC Server v1.00 (Dec 2019); NC software 340590–10 . . . 340595–10	Machine under study; provides the OPC UA server interface [OPC24, MAH09].
iTENDO^2	SCHUNK HSK-A63 Ø20×90; Ident.-Nr. 1484050; Sensor SW v2.1.0+; App v2.1 (17); Bluetooth 2.4 GHz	External vibration / process sensor system; integrated via ESP32-based bridge nodes [SCH25a, SCH25b].
Swarmy	Swarmy-V2 (Fraunhofer IZM sensor-actuator platform)	Battery-powered multi-sensor node (acceleration and environment); connected via UART serial link to the backend.
Edge PC	Intel industrial PC; Windows 11 Pro; Python 3.11 environment	Hosts sensor backends, OPC UA middleware, MongoDB, DataBridge, and the UI during tests.
Network	1 GbE LAN + Wi-Fi 6 (2.4/5 GHz)	Connectivity between machine, edge host, and AAS infrastructure in the EMOTION laboratory setup [IPK25].

4.2.2 Software and frameworks

Table 3 provides an overview of the software components and framework versions used throughout the implementation and evaluation environment. It covers backend services, middleware, database systems, frontend technologies, and containerization tools that together form the runtime infrastructure of the proposed integration architecture. Listing explicit versions ensures reproducibility of the setup and allows the reported results to be interpreted in the context of the employed software stack.

Tab. 3: Software and frameworks: versions used in the implementation and evaluation environment.

Software	**Version**	**Role / Notes**
Python	3.11.8	Runtime for the FastAPI middleware and sensor backends.
FastAPI (middleware)	0.119.1	OPC UA snapshot and health API for downstream integration [FAS25].
Uvicorn	0.38.0	ASGI server for FastAPI.
python-opcua (freeopcua)	0.98.13	OPC UA client library used by the middleware [OPC24, MAH09].
Quart (sensor backend)	0.19.6	REST/WebSocket services for Swarmy and ITENDO^2 backends [QUA25, RFC11].
MongoDB Server	6.0.18	Time-series persistence for sensor data and session metadata [MON21].
MongoDB Python driver	4.8.0	`pymongo` client for database operations.
Node.js	20.16.0	JavaScript runtime for the frontend toolchain.
npm	10.8.1	Node package manager (frontend dependencies).
React	18.3.1	Monitoring user interface [REA25].
Vite	5.3.1	Frontend development server and build tool.
Chart.js	4.4.4	Live data visualisation [CHA25].
Docker Engine	28.1.1	Container runtime (Docker Desktop 4.41.2) [MER14].
Docker Compose (v2)	2.35.1	Multi-service orchestration for reproducible deployment.
Eclipse BaSyx DataBridge	1.0.0-milestone-08	Configuration-driven HTTP-to-AAS publication using JSONata transforms [BAS25, JSO25, PLA23].
Eclipse BaSyx AAS Server/Registry	1.0.0-milestone-08	Hosting and discovery of AAS shells and submodels [BAS25, PLA23].
AAS package/editor	1.0.0	AAS authoring and inspection (e.g., AASX Package Explorer) [PLA23].
JSONata	1.8.3	Transformation language used in DataBridge pipelines [JSO25].
Git	2.45.1	Version control (commit IDs archived with the project).

4.3 References, Configuration, and Reproducibility

Configuration artefacts (DataBridge JSON, middleware configuration, Dockerfiles, and MongoDB indexes) and the relevant commit IDs for UI and backend components are archived with this thesis to support reproducibility (see Chapter 6 for measurement procedures). Container-

isation follows established practices for consistent deployment across environments [MER14]. AAS artefacts and endpoints conform to the published AAS metamodel and associated profiles [PLA23, BAS25].

4.3.1 Licensing and third-party notices

Open-source tools are used under their respective licences. Commercial components are referenced by model and version only. Licence texts and third-party notices are stored alongside the archived source tree.

4.3.2 Versioning note

All framework versions, Python package lists, and build manifests (e.g., `requirements.txt`, `package.json`) are included in the project archive. This enables recreation of the software environment either via Docker or on compatible systems with minimal manual configuration [MER14].

Role of the author. Within the EMOTION project, the author was responsible for the design and implementation of the data acquisition and integration pipeline described in this thesis. This includes the integration of external sensors, the implementation of backend communication services, the development of a unified user interface for live monitoring, and the mapping of selected OPC UA machine signals into AAS submodels hosted via Eclipse BaSyx [BAS25]. Tasks related to machine control, sensor hardware development, or advanced data analytics were not part of the author's responsibilities.

Boundary conditions. All experiments and evaluations presented in later chapters are based exclusively on data obtained from the described laboratory setup. Only non-confidential technical data and system behavior observations are reported. No personal data were processed, and all measurements were conducted on technical systems under controlled conditions. The described environment defines the boundary within which the results of this thesis are valid and provides the basis for interpreting the design decisions and evaluation outcomes presented in the following chapters.

5 System Design and Implementation

5.1 Overall Architecture and Methodology

5.1.1 Architectural Principles and Methodological Approach

The system design follows a structured engineering methodology that combines targeted preparatory analysis with iterative implementation in order to realize a dependable, standards-aligned data pipeline for an industrial machining testbed [TAO19]. From a data-engineering perspective, the approach is guided by established principles for ingesting, transporting, and persisting continuously generated time-series data under realistic runtime conditions [KLE17, AKI18].

The overall architecture is based on a clear separation of concerns. Data acquisition, back-end processing, semantic integration, and visualization are implemented as loosely coupled components, enabling reproducibility, robustness, and extensibility. Containerization is used throughout to ensure consistent deployment across development and evaluation environments [MER14].

Two complementary implementation tracks are pursued. First, external sensor systems are integrated to provide additional process- and condition-related measurements that are not available via the machine control. Second, selected machine-internal signals are accessed via OPC UA and embedded into a semantically structured Digital Twin representation using the Asset Administration Shell (AAS) and Eclipse BaSyx [OPC24, MAH09, AAS20, BAS25]. Advanced analytics, such as condition monitoring or predictive maintenance, are intentionally excluded and considered only as potential future extensions [TAO19].

Figure 1 provides an overview of the overall system architecture and the interaction between the main components of the implemented data integration pipeline. External sensor systems (Swarmy and ITENDO^2) deliver telemetry data to dedicated backend services, where measurements are normalized and prepared for further processing. In parallel, machine-internal signals are accessed from the Exeron HSC600 via OPC UA and retrieved by a middleware component that exposes a snapshot-based HTTP interface. Sensor data are streamed to the monitoring user interface via WebSocket connections for near-live visualization, while selected machine-side signals are semantically mapped into an Asset Administration Shell using Eclipse BaSyx. All acquired data are persistently stored in MongoDB time-series collections, providing a consistent basis for later analysis and evaluation.

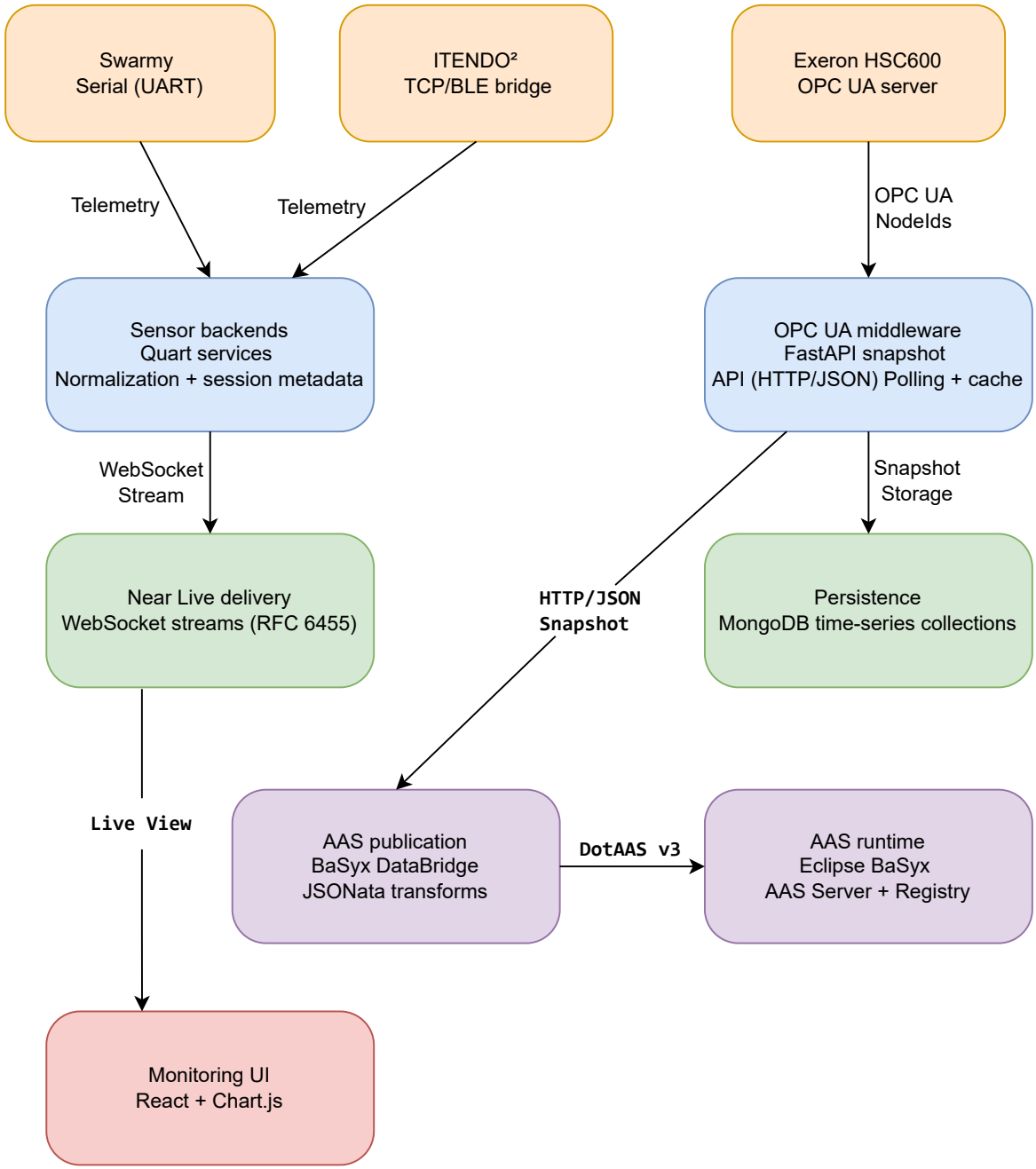


Fig. 1: Overall system architecture of the implemented monitoring and integration pipeline.

5.1.2 Implementation Process Stages

Figure 2 illustrates the logical data flow and the staged implementation approach used to realize the monitoring and AAS integration pipeline. The diagram highlights how heterogeneous input sources are progressively processed through acquisition, normalization, streaming, persistence, and semantic integration steps before being exposed to the user interface and validated through testing. The staged structure emphasizes separation of concerns and reflects the incremental implementation strategy followed during system development.

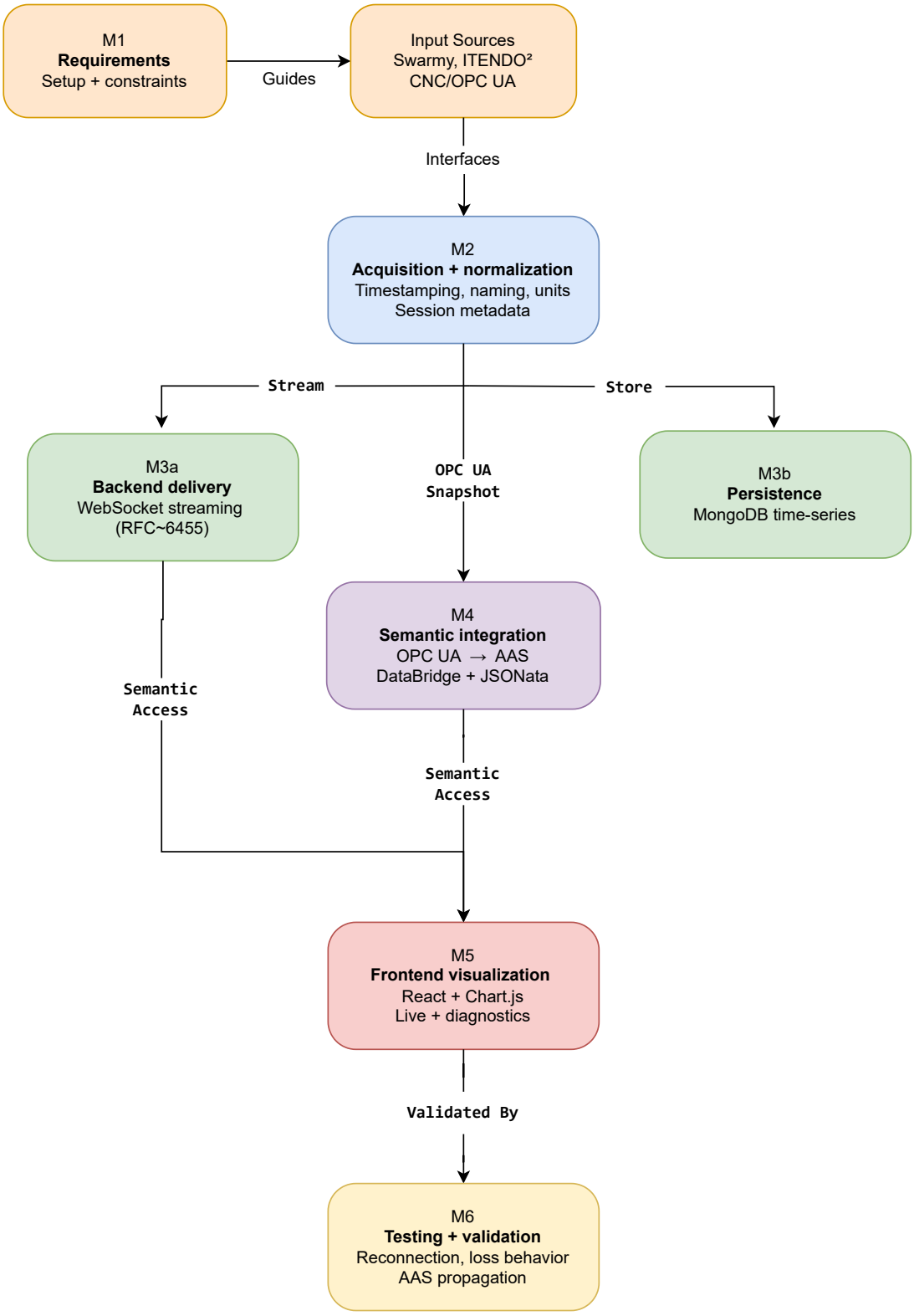


Fig. 2: Logical data flow and processing stages of the implemented monitoring and AAS integration pipeline.

The implementation process is structured into the following stages:

M1. Requirements and system understanding. The machining environment, available

OPC UA signal space, and the characteristics of the external sensor systems are analyzed within the EMOTION project context at Fraunhofer IPK [IPK25]. Requirements focus on continuous data acquisition with robust reconnection behavior, bounded packet loss, persistent time-series storage, stable user-interface performance, and semantic representation of selected machine signals in the AAS [PLA23, IDT24].

M2. Data acquisition and normalization. External sensor data and machine-internal signals are acquired through protocol-specific interfaces. At the acquisition layer, lightweight normalization is applied, including timestamp alignment, channel naming, unit conversion, and session metadata, to ensure consistent downstream handling [KLE17].

M3. Backend delivery and persistence. Live data are delivered to the frontend via a unified WebSocket interface in accordance with RFC 6455 [RFC11]. Backend services are implemented using FastAPI and Quart [FAS25, QUA25] and persist all samples in MongoDB time-series collections optimized for recent-window queries and controlled data growth [MON21].

M4. Semantic integration of machine signals. Selected OPC UA signals are mapped into AAS submodels and exposed via Eclipse BaSyx [AAS20, BAS25]. Configuration-driven publication is realized using the BaSyx DataBridge with JSONata-based transformations [JSO25]. External sensor data are intentionally excluded from AAS modeling, consistent with the defined scope [PLA23].

M5. Frontend visualization. A browser-based user interface provides a unified view of live telemetry, connection states, and session-level diagnostics. The UI is implemented using React and Chart.js to ensure responsive and low-latency visualization [REA25, CHA25].

M6. Testing and validation. Functional and connectivity tests validate robust acquisition, reconnection handling, representative packet-loss behavior, and correct propagation of semantic updates through the AAS infrastructure [AKI18]. Detailed evaluation results are presented in Chapter 6.

5.1.3 Goals, Scope, and Intended Outcome

Goal. The goal of this work is to realize a modular and reproducible system architecture that integrates heterogeneous data sources, provides unified live monitoring, and embeds selected machine signals into a semantically structured Digital Twin representation using the AAS [AAS20, PLA23].

Scope. The scope includes protocol-specific acquisition, unified WebSocket delivery, MongoDB-based time-series persistence, and OPC UA $\rightarrow$ AAS integration via Eclipse BaSyx [OPC24, BAS25, MON21]. Explicitly excluded are predictive analytics, machine learning methods, full semantic modeling of external sensors, and exhaustive performance benchmarking [TAO19].

Outcome. The outcome is a validated prototype architecture that forms a reliable foundation for subsequent sensor-specific integrations and semantic extensions. The resulting implementation supports future Digital Twin–based applications within the EMOTION project while remaining clearly scoped and reproducible [IPK25, EMO25].

5.2 Swarmy Sensor Integration

5.2.1 Swarmy Sensor System

Swarmy is a battery-powered multi-sensor node used in the EMOTION testbed to capture condition-related measurements that are not available through the machine control system. It provides (i) continuous three-axis acceleration measurements in an acceleration mode, and (ii) discrete environmental readings, including temperature, relative humidity, and battery voltage, as well as High-G impact events in passive operating modes. In the configuration considered in this thesis, Swarmy is connected to the edge host exclusively via a wired Universal Asynchronous Receiver–Transmitter (UART) serial interface; no Bluetooth Low Energy (BLE) stack is required on the host side. This setup follows the reference testbed architecture established in the EMOTION research project at Fraunhofer IPK [IPK25].

Communication interface and data framing. The serial interface operates at `230400 8N1`. During active measurement phases, Swarmy emits binary acceleration frames that are separated by a fixed synchronization marker `0D 0A 00 00 00 00`. The marker enables resynchronization if bytes are dropped or partial frames occur, which is a common requirement in long-running serial streams [KLE17]. Each frame contains a sequence of (x, y, z) samples encoded as 16-bit signed integers in a device-specific byte order that requires byte swapping on the host side. A sensitivity factor of 4096 LSB$/g$ converts raw counts into physical acceleration values in units of gravitational acceleration (g).

During inactive phases, Swarmy periodically emits ASCII-encoded environmental measurements in the format `Temperature: <float> °C Humidity: <float> % Accu: <float> V`. In addition, High-G impact events are reported asynchronously using interrupt-style messages of the form `High G Interrupt! x: <int> y: <int> z: <int>`, where the transmitted integers represent axis-specific peak accelerations. These sparse events are handled separately from the continuous acceleration stream to preserve steady-state acquisition.

5.2.2 Integration and Implementation

This subsection describes the acquisition, processing, visualization, and persistence pipeline implemented for Swarmy within the overall system architecture introduced in Section 5. The integration follows a clear separation between control and historical access via REST interfaces and low-latency telemetry delivery via WebSocket connections, which is a common architectural pattern for interactive monitoring systems [AKI18].

Serial acquisition and parsing. On the backend, a dedicated serial reader continuously accumulates incoming bytes and segments the data stream using the defined synchronization marker. After basic validation of frame boundaries and length, acceleration samples are decoded, converted to physical units, and enriched with timestamps and session metadata. The implementation relies on non-blocking I/O and includes recovery mechanisms for incomplete or malformed frames, enabling robust long-running acquisition under laboratory conditions [KLE17].

ASCII-encoded environmental measurements are parsed using regular expressions and stored as individual time-series signals (Temperature, Humidity, Accu). High-G interrupt messages are treated as event records containing axis-specific peak values. All parsed measurements are prepared for persistence in MongoDB time-series collections [MON21].

Runtime services and interfaces. The Swarmy backend is implemented in Python using the `Quart` framework [QUA25]. REST endpoints are used for configuration, operating-mode selection, and historical data retrieval, while WebSocket channels provide near real-time telemetry in accordance with RFC 6455 [RFC11]. Table 4 summarizes the exposed runtime interfaces.

Tab. 4: Swarmy runtime services: REST and WebSocket endpoints.

Method/Path	Purpose	Notes
GET `/temperature-history`	Temperature series	`?limit&from&to` (ISO/epoch)
GET `/humidity-history`	Humidity series	same parameters
GET `/accu-history`	Battery voltage series	same parameters
GET `/high-g-history`	High-G events	newest first
GET `/mode`	Current operating mode	`{code,label}`
POST `/set-mode`	Apply operating mode	normalized mode identifier
POST `/start-acceleration`	Enable streaming	device command
POST `/stop-acceleration`	Disable streaming	device command
GET `/get-battery`	Battery status estimate	voltage and heuristic lifetime
POST `/packet-loss`	Persist loss metric	reported by UI
GET `/packet-loss-last`	Latest loss metric	quick status access
WS `/ws`	Live XYZ acceleration	$\sim$20 Hz, framed JSON
WS `/high`	Live High-G events	heartbeat or last event

User interface views. The user interface presents Swarmy data through three dedicated views, each tailored to a specific monitoring task and implemented using React and Chart.js [REA25, CHA25]:

- **Monitoring view**: device status (connection and mode), battery state, and packet-loss indicator; in processing mode, the view additionally visualizes live three-axis acceleration time series.

- **Storage view**: historical temperature, humidity, and battery voltage retrieved via REST queries.
- **Transport view**: tabular listing of High-G impact events with CSV export for further analysis.

Representative screenshots for all views are provided in Appendix 10, Figures 3–6.

Data persistence and monitoring. All Swarmy measurements are persisted in MongoDB using time-series collections [MON21]. Acceleration samples, environmental readings, and High-G events are stored in separate collections to support efficient time-window queries and stable dashboard refresh behavior [KLE17]. To make stream quality observable during operation, the frontend computes a packet-loss indicator over a rolling 30 s window based on deviations from the expected update rate. The metric is smoothed using an exponentially weighted moving average (EWMA) and periodically persisted, following common practice for monitoring continuous data streams [AKI18].

Data provenance. At the time of writing, the Swarmy sensors have not yet been mounted on the machine tool. The screenshots shown in Appendix 10, Figures 3–6 therefore originate from controlled bench tests. These experiments validate the end-to-end acquisition, delivery, visualization, and persistence pipeline and provide a verified basis for future machine-mounted measurements within the EMOTION project context [IPK25].

5.3 ITENDO^2 Sensor Integration

This section describes the integration of the ITENDO^2 vibration sensing system into the overall monitoring architecture introduced in Section 5. The implementation covers the complete data path from spindle- and table-mounted sensors to live visualization and persistent storage.

The realized system comprises:

1. two ESP32-based sampling nodes (table-mounted and spindle-mounted), built on the ESP32 microcontroller platform, that stream compact measurement frames over the Transmission Control Protocol (TCP) [SCH25a, SCH25b],
2. a Python backend implemented with `Quart` that manages TCP connections, session state, persistence, and REST/WebSocket interfaces [QUA25, MON21, RFC11], and
3. a browser-based frontend implemented with React and Chart.js for near real-time monitoring and session-based data inspection [REA25, CHA25].

5.3.1 System Overview

Each ITENDO^2 sensor node exposes a lightweight TCP server on port 3000. The backend maintains one TCP connection per sensor, decodes incoming frames, converts raw values into

physical acceleration units (g), and computes packet-loss metrics based on sequence counters embedded in the data stream. All measurements are persisted in MongoDB time-series collections optimized for high-frequency industrial data [MON21].

For live visualization, the backend publishes a reduced, chart-friendly data stream via WebSocket. Incoming packets are aggregated using a median-per-packet strategy and forwarded to the UI at approximately 30–33 Hz, balancing latency, bandwidth, and rendering stability [RFC11, AKI18].

Representative screenshots for all ITENDO^2 user-interface views are provided in Appendix 10, Figures 7–9.

5.3.2 Communication and Control

Each ESP-based sensor node accepts a small set of ASCII control commands over TCP:

- `connect` / `disconnect`: establish or release the BLE link to the ITENDO^2 sensor,
- `subscribe` / `unsubscribe`: start or stop continuous data streaming,
- `ping`: keep-alive message to detect idle or broken connections.

The backend mirrors these commands through REST endpoints exposed by the Quart application, enabling the frontend to control sensor state without direct TCP access [QUA25]. Table 5 summarizes the available backend interfaces.

Tab. 5: ITENDO^2 backend API: REST and WebSocket endpoints.

Method	**Path**	**Purpose**
POST	`/connect?sensor=itn_wzth|itn_spdl`	Establish BLE connection via ESP node.
POST	`/disconnect?sensor=...`	Close BLE connection and reset state.
POST	`/start?sensor=...`	Start data streaming (`subscribe`).
POST	`/stop?sensor=...`	Stop data streaming (`unsubscribe`).
GET	`/sensor-status`	Current connection and runtime flags.
GET	`/sessions`	List recorded sessions with filters.
GET	`/sessions/export`	Export session data as CSV.
GET	`/last-session`	Retrieve most recent session.
WS	`/ws`	Live monitoring data stream.

WebSocket payload. To minimize bandwidth and rendering overhead, each WebSocket message contains timestamp–value pairs derived from aggregated sensor packets. Timestamps are expressed as relative seconds from session start and mapped to absolute time in the UI for sliding-window visualization. An example payload is shown in Listing 1.

Example payload (WebSocket framing according to RFC 6455 [RFC11]) is shown in Listing 1.

```
{
  "table_data": [{"timestamp": 0.00, "value": 0.98}, {"timestamp": 0.03, "value": 1.00}, ...],
  "spindle_data": [{"timestamp": 0.00, "value": 1.02}, {"timestamp": 0.03, "value": 1.01}, ...],
  "table_range": [7.4, 16.8 ],
  "spindle_range": [7.3, 16.7 ]
}
```

Listing 1: ITENDO^2 monitoring WebSocket payload (reduced stream): timestamp value pairs per sensor and Y-axis range hints.

5.3.3 Backend Architecture

The backend is implemented in Python using `Quart` and supervises two long-lived TCP client connections, one per sensor. Incoming messages are parsed as either battery updates or data packets containing a monotonically increasing packet counter and packed channel values.

Packet loss is computed from gaps in the counter field and stored both as per-sensor metrics and as aggregated KPIs at the session level. MongoDB time-series collections are used for efficient ingestion and querying of high-frequency data streams [MON21, KLE17].

Session lifecycle management follows a clear model: sessions are created upon the first `subscribe` command and closed after the final `unsubscribe`. A dedicated writer thread batches database inserts to avoid blocking TCP readers, following established practices for back-pressure management in streaming systems [AKI18].

Reliability and lifecycle handling. The service registers signal handlers for graceful shutdown and ensures consistent cleanup of sockets and background threads. TCP keep-alive messages and periodic `ping` commands mitigate idle disconnects. Upon any connection loss, the backend updates runtime flags so that the UI always reflects a consistent system state [QUA25].

5.3.4 Frontend Visualization

The monitoring view renders two synchronized line charts with adaptive time windows and dynamic Y-axis scaling centered around zero. A unified toolbar displays per-sensor connection state and provides a single control to start or stop both data streams.

The Sensor Data view aggregates completed sessions, displays average packet-loss KPIs for table and spindle sensors, and supports CSV export for offline analysis. The frontend is implemented with React and Chart.js to ensure responsive rendering and stable real-time updates [REA25, CHA25].

5.3.5 Results and Observed Metrics

During laboratory tests, the UI rendered smoothly at approximately 30 Hz with WebSocket payloads of only a few kilobytes per update. Observed packet-loss rates were typically below 3 % for the table sensor and below 2 % for the spindle sensor, as reflected in the session summaries shown in Figure 9. This confirms that the chosen architecture supports reliable and observable data acquisition under representative operating conditions [AKI18].

5.3.6 Configuration Notes

Networking and database parameters are configured via environment variables, including sensor host addresses, TCP ports, and MongoDB connection settings. Indexes are created idempotently at startup, and runtime flags are reset to safe defaults to ensure predictable behavior after restarts [MON21].

5.3.7 Control Flow and Session Coordination

The ITENDO^2 monitoring workflow follows a clear separation of responsibilities between the frontend user interface and the backend services.

The frontend orchestrates user-initiated control actions, including BLE connection and disconnection as well as subscription and unsubscription to continuous data streams. These actions are exposed to the user through unified controls, allowing both sensors to be started or stopped simultaneously while still supporting independent operation.

The backend is responsible for maintaining session consistency and data integrity. It supervises the TCP connections to the ESP-based bridge nodes, tracks per-sensor runtime state, and ensures that session lifecycles remain well-defined even when only one sensor is active. In particular, sessions are created on the first successful `subscribe` command and are only closed once the final active sensor is unsubscribed. This design avoids fragmented sessions and guarantees consistent aggregation of packet-loss metrics and persisted samples.

The resulting control flow ensures predictable behavior during partial disconnects, staggered start/stop actions, and transient communication failures. A schematic overview of the implemented control-flow logic is provided in Appendix 10, Figure 10.

5.4 OPC UA-Based Asset Administration Shell Integration (HSC600)

This section describes the software pipeline used to integrate selected machine signals from the Exeron HSC600 into a semantically structured Digital Twin representation based on the Asset Administration Shell (AAS). Machine-internal signals are read from the OPC UA server provided by the HSC600 control [OPC24, MAH09], exposed through a lightweight FastAPI middleware [FAS25], and forwarded into Eclipse BaSyx submodels using the configuration-driven Eclipse BaSyx DataBridge with JSONata transformations [BAS25, JSO25]. The implementation

emphasizes robustness (cyclic polling with automatic reconnect), transparent data exchange (HTTP/JSON), and reproducible deployment through containerization [MER14]. Advanced analytics and control logic are intentionally out of scope.

5.4.1 Architecture Overview

The integration architecture consists of two decoupled components:

- **OPC UA middleware (FastAPI).** A background poller establishes a secure OPC UA session, periodically reads a configured set of NodeIds, and caches the most recent values together with source and server timestamps [OPC24]. The middleware exposes a small HTTP API, including endpoints for health monitoring, node metadata, individual values, and a normalized snapshot view for downstream consumption [FAS25].
- **AAS publication layer (Eclipse BaSyx DataBridge).** A timer-driven HTTP consumer retrieves the middleware snapshot. JSONata expressions extract the required signal values, which are then written into designated AAS submodel elements via their `idShortPath` using the DotAAS V3 API [BAS25, JSO25, PLA23].

This separation keeps machine communication concerns isolated from semantic modeling and AAS publication.

5.4.2 Middleware Implementation (FastAPI)

The middleware hosts a background polling thread that connects to the OPC UA server and performs cyclic reads at a fixed interval of 3 s (configurable). Connection status and polling health are exposed via `/health`, enabling supervision by the DataBridge and external diagnostics tools [FAS25].

The poller is responsible for:

1. establishing and maintaining a secure OPC UA session,
2. reading a predefined set of NodeIds at the configured interval,
3. updating an in-memory cache of values and timestamps for HTTP access,
4. detecting failures and exposing the current connection state.

Listing 2 shows a simplified excerpt of the polling loop. Additional implementation details (snapshot schema, mapping configuration, and client reconnect logic) are provided in Appendix 10.

```
1 # app/main.py (excerpt)
2 POLL_INTERVAL_SEC = 3
3 _cache, _last_update, _connected = {}, None, False
4
5 def _poll_loop():
```

```
    global _last_update, _connected
    manager.connect()
    _connected = True
    ids = nodeids()
    while not _stop_event.is_set():
        try:
            result = manager.read_nodes(ids)
            with _cache_lock:
                _cache.clear()
                _cache.update(result)
                _last_update = datetime.utcnow()
            _connected = manager.is_connected()
        except Exception:
            _connected = False
        _stop_event.wait(POLL_INTERVAL_SEC)
```

Listing 2: Background polling loop in the OPC UA middleware: periodic NodeId reads, cache updates, and connection state tracking.

Exposed HTTP endpoints. The middleware exposes the endpoints summarized in Table 6; the DataBridge consumes `/snapshot` as a normalized, single-call view of the current signal set.

Tab. 6: OPC UA middleware HTTP API.

Method	Path	Purpose
GET	`/health`	Connection status, poll interval, last update time.
GET	`/nodes`	Mapping between friendly names and OPC UA NodeIds.
GET	`/values`	Cached values keyed by NodeId.
GET	`/value/{nodeid}`	Single NodeId lookup.
GET	`/prop/{name}`	Read value by friendly signal name.
GET	`/snapshot`	Aggregated view for DataBridge consumption.

5.4.3 Signal Selection and Node Mapping

Machine signals are selected explicitly via a configuration mapping from human-readable names to OPC UA NodeIds. This improves traceability while preserving the original OPC UA addressing scheme [OPC24, MAH09]. A representative excerpt of the mapping is provided in Appendix 10, Listing 6.

5.4.4 AAS Publication (Eclipse BaSyx DataBridge)

The Eclipse BaSyx DataBridge is configured without code changes: an HTTP consumer retrieves the middleware snapshot, JSONata transforms select the relevant fields, and an AAS

sink writes values into submodel elements [BAS25, JSO25, PLA23]. Representative configuration excerpts (consumer, transformer, sink, timer, and route) are provided in Appendix 10, Listings 8–12.

5.4.5 Containerization (Docker)

Middleware and DataBridge are deployed as containers to ensure a consistent runtime environment and reproducible setup [MER14]. The Docker Compose and Dockerfile excerpts are provided in Appendix 10, Listings 13 and 14.

5.4.6 AAS Visualization in Eclipse BaSyx

The resulting AAS representation can be inspected using the Eclipse BaSyx UI. Representative screenshots of mapped properties and selected server diagnostics are provided in Appendix 10, Figures 11–17.

5.4.7 Notes and Configuration

- **Security.** Certificates and keys are loaded by the OPC UA client; credentials are configured via environment variables or a secrets file (not committed to version control) [OPC24].
- **Timestamps.** The API returns ISO 8601 UTC timestamps; the effective Eclipse BaSyx element update cadence equals the bridge timer (1 Hz in this configuration).
- **Extensibility.** Adding a new machine signal requires only: (i) a new entry in `NODES`, (ii) one JSONata file, (iii) one sink entry, and (iv) a route entry [BAS25, JSO25].

6 Evaluation and Testing

6.1 Evaluation Objectives and Success Criteria

The evaluation assesses whether the implemented monitoring system meets the requirements of this thesis with respect to (i) correct acquisition of sensor and machine signals, (ii) reliable delivery to the frontend, (iii) persistent storage for later analysis, and (iv) correct semantic publication of selected machine states via the Asset Administration Shell (AAS). The scope therefore covers the complete data path from Swarmy (serial), ITENDO^2 (TCP), and HSC600 machine signals (OPC UA) [OPC24, MAH09] to WebSocket-based UI streaming [RFC11], MongoDB time-series persistence [MON21], and AAS exposure through Eclipse BaSyx [PLA23, BAS25].

To make the evaluation outcomes explicit and traceable, success criteria are defined upfront and are assessed against the reported results (Table 12). Beyond functional correctness, the evaluation focuses on operational behavior relevant to live monitoring, including connection stability, packet-loss indicators for ITENDO^2, continuity of stored samples, recovery behavior after induced failures, and fidelity of the AAS mapping [AKI18, KLE17, PLA23].

Tab. 7: Success criteria under nominal operation and corresponding target conditions.

Criterion ID	**Description and Target**
C1 – Functional coverage	Live sensor streams are visible in the UI, and selected AAS elements update correctly and are readable through the AAS infrastructure [BAS25].
C2 – Reliability under nominal load	ITENDO^2 packet-loss indicators remain low (target: $\leq$ 3 %) and do not lead to prolonged visualization gaps [AKI18]. Swarmy acceleration telemetry remains stable at its configured update rate (approximately 20 Hz) with only short, bounded gaps [RFC11].
C3 – Recovery behavior	After induced failures, the system resumes streaming without manual reconfiguration; time-to-first-valid-message remains bounded (target: $\leq$ 10 s) [OPC24].
C4 – Continuity in storage	Stored series show no uncontrolled gaps beyond short transient interruptions; continuity checks indicate low missing/expected ratios under nominal operation [MON21].
C5 – Semantic correctness	Mapped AAS element paths match the intended submodel structure and values match the middleware snapshot; elements are addressable via the expected `idShortPath` [PLA23, BAS25].
C6 – Storage usability	Recent-window queries (last 1–5 min) complete within a bounded latency when indexes are present (target: $\leq$ 150 ms) [MON21, KLE17].

6.2 Testbed

All experiments are conducted in the EMOTION laboratory setup using the Exeron HSC600 machine tool, with the external sensor systems Swarmy and ITENDO^2. The backend provides REST endpoints for configuration and historical retrieval as well as WebSocket streams for live visualization [RFC11]. Time-series data are stored in MongoDB 5.0 time-series collections [MON21]. Selected machine signals are read from the HSC600 OPC UA server and published into AAS submodels served by Eclipse BaSyx [OPC24, BAS25, PLA23]. Hardware, operating system, network setup, and software versions are documented in Chapter 4 to support repeatability.

6.3 Metrics and Instrumentation

The evaluation uses quantitative indicators where feasible, complemented by targeted qualitative checks (e.g., UI behavior). The following metrics are collected, together with their sources and computation methods:

- **Connectivity and uptime.** Connection state is observed for: ITENDO^2 (TCP socket + BLE state per sensor), Swarmy (serial link), OPC UA session, and WebSocket sessions. States are sampled periodically and logged for fault-injection experiments [OPC24, RFC11].
- **Packet loss (ITENDO^2).** Packet loss is derived from gaps in the per-sensor packet counter and smoothed using an exponentially weighted moving average (EWMA, $\alpha = 0.25$) to avoid overreacting to transient outliers [AKI18, KLE17].
- **Effective update rate (UI).** The frontend update cadence is checked against the intended stream rates: approximately 30–33 Hz for the reduced ITENDO^2 WebSocket stream, and approximately 20 Hz for Swarmy acceleration telemetry (as configured in the Swarmy service) [RFC11].
- **Reconnection time.** Recovery time is measured from an induced disconnect (network, sensor service, or OPC UA session) to the first valid update visible at the UI/WebSocket level [AKI18].
- **Continuity of stored samples.** For representative sessions, continuity is assessed from stored timestamps (expected vs. missing samples and gap durations). The computation is performed directly over MongoDB time-series documents [MON21].
- **AAS mapping correctness.** For selected mapped machine signals, correctness is verified by comparing middleware values (e.g., `/prop/{name}` or `/snapshot`) with the corresponding submodel element value in Eclipse BaSyx, and by confirming that the element is addressable via the expected `idShortPath` and visible through the configured AAS infrastructure [PLA23, BAS25].

- **Storage behavior (MongoDB).** Insert success is checked indirectly through monotonic growth of documents and absence of missing intervals under nominal conditions. Query latency is measured for fixed recent time windows with indexes present [MON21, KLE17].

Helper queries (MongoDB). To make continuity and query latency checks repeatable, compact analysis helpers are used. Listing 3 illustrates a continuity check based on timestamp gaps, and Listing 4 shows a small recent-window query used as a micro-benchmark [MON21].

```
def continuity(coll, session_id, expected_hz):
    cur = coll.find({"session_id": session_id}).sort("timestamp", 1)
    ts = [doc["timestamp"] for doc in cur]
    if len(ts) < 2:
        return {"expected": 0, "missing": 0, "gaps": []}
    dt = 1.0 / expected_hz
    gaps, missing = [], 0
    for a, b in zip(ts, ts[1:]):
        gap = (b - a).total_seconds()
        if gap > 1.5 * dt: # tolerant threshold
            miss = int(round(gap / dt)) - 1
            missing += max(miss, 0)
            gaps.append({"start": a, "end": b, "sec": gap, "missing": miss})
    expected = int(round((ts[-1] - ts[0]).total_seconds() * expected_hz))
    return {"expected": expected, "missing": missing, "gaps": gaps}
```

Listing 3: Continuity checker (pseudo-Python): estimate missing samples and gap durations from stored timestamps for a given session.

```
t0 = datetime.utcnow() - timedelta(minutes=2)
tic = time.perf_counter()
docs = list(db.sensor_itn_spdl_data.find({"timestamp": {"$gte": t0}}).limit(20000))
lat_ms = 1000 * (time.perf_counter() - tic)
```

Listing 4: Recent window query (micro-benchmark): latency measurement for retrieving the last two minutes of samples.

6.4 Experiments

The experiments are designed to cover nominal operation, stress conditions, and recovery behavior. Each experiment is executed multiple times where applicable to avoid single-run artifacts.

E1. Baseline connectivity and timestamping. Verify stable connectivity to Swarmy (serial), ITENDO^2 (TCP/BLE), and OPC UA (HSC600). Confirm that timestamps are recorded in UTC and are consistent across services [OPC24].

E2. WebSocket delivery and schema validation. Validate that WebSocket messages arrive continuously, include the expected fields (keys, units, ranges), and do not cause parsing errors or UI stalls [RFC11].

E3. Persistence and continuity. Verify insertion into MongoDB time-series collections and compute continuity metrics for representative sessions (missing/expected ratio and maximum gaps) [MON21].

E4. AAS mapping verification. For each mapped signal, compare middleware values to the corresponding AAS submodel element in Eclipse BaSyx and verify correct addressing via `idShortPath` [PLA23, BAS25].

E5. Stress and burst behavior. Induce higher activity (e.g., increased vibrations or controlled motion) and observe changes in packet-loss indicators, UI update behavior, and continuity metrics [AKI18].

E6. Fault injection and recovery. Induce short failures (e.g., temporary network interruption, restarting sensor services, terminating OPC UA sessions) and measure time-to-recover until a valid stream update is observed [OPC24, BAS25].

6.5 Results

Tables 8–11 summarize representative results from the conducted experiments. The values capture nominal behavior and selected stress/fault-injection scenarios.

Tab. 8: ITENDO^2 packet loss and reconnection: representative scenarios with per-sensor loss and recovery time (time to first valid UI update).

Scenario	Table Loss [%]	Spindle Loss [%]	Reconn. [s]	Notes
Nominal steady	0.00	0.00	–	Stable baseline with continuous streaming.
Sensor motion (mild)	0.45	0.15	2–3	Minor transient loss under movement; stream resumed quickly.
Stress burst	7.95	2.06	4–5	Increased transient loss under high activity; recovery remained bounded.
Network pull (5 s)	2.76	0.52	5–6	Controlled interruption; automatic recovery after connectivity returned.

Tab. 9: Continuity metrics (representative sessions): expected versus missing samples and maximum gap duration derived from stored timestamps.

Stream	Expected	Missing	Missing [%]	Max Gap [s]	Notes
ITENDO table	100 000	2 760	2.76	0.9	Missing ratio consistent with packet-loss KPI in the session overview.
ITENDO spindle	100 000	520	0.52	0.3	Stable behavior with short maximum gaps under the tested conditions.
Swarmy acceleration	996	48	4.86	1.1	Bounded gaps observed during live streaming; matches UI loss indicator.

Tab. 10: AAS mapping verification: selected machine signals with NodeIds, target `idShortPath`, registry presence, and value consistency.

Name	NodeId	idShortPath	Registry	Match
Chuck2IsClosed	ns=4;i=9	Chuck2IsClosed.Value	✓	✓
Chuck2IsOccupied	ns=4;i=10	Chuck2IsOccupied.Value	✓	✓
MaxArrayLength	ns=0;i=11702	MaxArrayLength.Value	✓	✓
SessionTimeoutCount	ns=0;i=2281	SessionTimeoutCount.Value	✓	✓

Tab. 11: Swarmy stream KPIs: packet-loss indicator, effective update rate, reconnection time, and gap characteristics under representative scenarios.

Scenario	Loss [%]	Rate eff. [Hz]	Reconn. [s]	Max Gap [s]	Notes
Nominal steady	4.86	20	–	0.9	Stable telemetry at configured rate; no prolonged gaps observed.
Motion (mild)	5.10	19	1–2	1.0	Slight increase in transient loss under movement.
Stress burst	19.27	16	3–4	1.3	Increased transient loss during high activity; bounded recovery.
Serial unplug (5 s)	20.05	16	5–6	1.5	Controlled disconnect; stream resumed after link restoration.

Tab. 12: Assessment of the results against the predefined success criteria (Table 7).

Criterion ID	Evidence in this chapter	Achieved
C1	Live UI streams for Swarmy and ITENDO^2; selected AAS elements visible and updating (Table 10).	✓
C2	Nominal ITENDO^2 packet loss within target; Swarmy telemetry maintained configured update rate with bounded gaps (Tables 8, 11).	✓
C3	Recovery after induced faults bounded to a few seconds and below 10 s (Tables 8, 11).	✓
C4	Stored time-series continuity shows low missing ratios and bounded gaps (Table 9).	✓
C5	Correct `idShortPath` addressing and value consistency between middleware and AAS (Table 10).	✓
C6	Recent-window query latency remained within the defined bound under indexed collections (measured as described in Section 6).	✓

6.6 Interpretation of Results

Overall, the results indicate that the system fulfills the functional requirements and remains operationally usable under both nominal and stress conditions. For ITENDO^2, nominal packet loss remained low and recovery after induced interruptions occurred within a few seconds, which is consistent with expected behavior for live streaming systems that prioritize fast state convergence over strict losslessness [AKI18, KLE17]. Continuity checks on stored sessions confirm that gaps remained bounded and that the persisted datasets are suitable for subsequent analysis [MON21].

Swarmy exhibited higher variability under stress scenarios. This is reflected by both the packet-loss indicator and the effective update rate; however, reconnection behavior remained bounded in the tested cases, and environmental channels remained continuously available during passive modes.

For the AAS integration, all tested signals were readable in Eclipse BaSyx and matched the middleware's snapshot values, indicating correct bridging and correct addressing of submodel elements through the configured `idShortPath` mappings [PLA23, BAS25]. This supports the correctness of the overall semantic publication pipeline.

6.7 Threats to Validity

The experiments were conducted under controlled laboratory conditions on a single machine and a fixed network setup. Packet loss and reconnection behavior can vary under different radio environments (BLE interference), host load, or firmware versions [AKI18]. Metric accuracy is affected by (i) the sampling granularity used for connection-state logging, (ii) smoothing in loss indicators (EWMA), and (iii) the limited duration and number of stress runs. These risks were

mitigated by repeating key experiments and cross-validating results across UI indicators and MongoDB continuity checks [MON21].

Construct validity is limited by the use of derived indicators (e.g., inferred missing samples and effective update rates) rather than dedicated time-synchronization instrumentation across all components. Nevertheless, consistent agreement between UI behavior, backend logs, database contents, and AAS reads provides strong internal consistency for the evaluated pipeline [BAS25, PLA23].

7 Discussion

This chapter discusses the implemented monitoring and integration system with respect to the thesis goals defined in Chapter 3. The focus lies on interpreting the achieved functionality, the observed operational behavior, and the resulting engineering trade-offs. Rather than repeating the evaluation results from Chapter 6, the discussion consolidates what the results imply for dependable data integration in an industrial machining testbed and how the design choices align with established practices in streaming and data-intensive system design [KLE17, AKI18].

End-to-end integration in a heterogeneous testbed. A central outcome of this thesis is the realization of an end-to-end data path that integrates machine-internal signals (OPC UA) and external sensor streams (serial, TCP) into a unified monitoring and storage pipeline. This directly addresses the practical gap between data availability and dependable integration described in the problem statement: although the individual technologies are well established, bringing them together in a consistent, operationally usable architecture requires explicit handling of protocol differences, timing behavior, and failure modes [KLE17, AKI18]. The implemented separation of acquisition services, WebSocket-based live delivery, and time-series persistence provides a modular structure that supports incremental extension without coupling all components into a single monolith [RFC11, MON21, MER14].

Robust acquisition and runtime observability. The evaluation indicates that the system remains usable under nominal and stress conditions, while also making non-ideal behavior observable through runtime indicators. This reflects a pragmatic interpretation of “dependable operation” for real-time monitoring: the primary objective is not strict losslessness, but fast recovery, transparency of data quality, and stable downstream behavior [AKI18, KLE17]. For ITENDO^2, packet-loss estimation based on sequence counters and session-level KPIs supports interpretation of stored datasets, particularly when short gaps are unavoidable due to BLE/TCP bridging behavior and transient connectivity events. For Swarmy, the combination of framing-based resynchronization and UI-side loss estimation provides an operational signal of stream quality even when the upstream device does not provide explicit sequence metadata. Overall, the system demonstrates that explicit instrumentation (connection flags, loss indicators, and session metadata) is essential to interpret time-series data correctly in practice [AKI18, MON21].

Trade-offs in live visualization versus storage fidelity. A key engineering trade-off is the difference between (i) full-resolution ingestion and storage and (ii) reduced-rate streaming for visualization. The monitoring UI benefits from an aggregation strategy that reduces payload size and stabilizes rendering, while persistent storage retains the underlying samples for later inspection and analysis. This mirrors common architectural patterns in streaming systems where separate “serving” and “storage” paths are required to balance latency, throughput, and usability [AKI18, KLE17]. In the implemented solution, WebSocket delivery provides low-latency

updates with a bounded message size, while MongoDB time-series collections provide an efficient persistence mechanism for high-frequency data and typical recent-window queries [RFC11, MON21].

Semantic publication of machine signals via AAS. Another main contribution is the standards-oriented mapping of selected OPC UA machine signals into an Asset Administration Shell representation hosted by Eclipse BaSyx. This addresses the thesis goal of semantic publication while keeping the mapping effort manageable through configuration-driven transformation and routing [AAS20, PLA23, BAS25, JSO25]. The chosen approach (OPC UA polling middleware + DataBridge publication) deliberately separates machine communication from semantic modeling concerns. This reduces complexity at each layer and enables independent debugging and validation of the OPC UA access path, the snapshot schema, and the AAS write path. The evaluation confirms that the mapped values are visible through the AAS infrastructure and addressable through consistent `idShortPath` elements, which supports interoperability and reuse within the broader Digital Twin context [PLA23, BAS25].

Scope decision: excluding external sensors from formal AAS modeling. External sensor streams are intentionally integrated at the monitoring and storage layers rather than embedded into the AAS in this implementation phase. This decision reflects two considerations. First, integrating heterogeneous sensor payloads into a consistent semantic model requires additional domain modeling effort (units, semantics, operating modes, metadata) that goes beyond the defined scope. Second, the thesis focuses on dependable acquisition and interpretable runtime behavior; enforcing full semantic integration for all sensors would have increased complexity without being necessary to demonstrate the pipeline's operational correctness [AAS20, IDT24, PLA23]. Nevertheless, the implemented architecture preserves contextual association between sensor data and the machine asset, providing a realistic foundation for future semantic extensions guided by existing submodel templates and profiles [IDT24].

Reproducibility and deployment implications. Containerization and explicit documentation of tools and versions support reproducible deployment and reduce environment-specific errors during integration and evaluation [MER14]. In industrial R&D environments with evolving toolchains, this is not merely a convenience but a prerequisite for stable operation, repeatable experiments, and maintainable handover to project teams. The use of standard web interfaces (HTTP/JSON and WebSockets) further improves portability and interoperability across components and platforms [FAS25, QUA25, RFC11].

Limitations and generalizability. The results are derived from a single laboratory testbed with a fixed network setup and specific sensor firmware behavior. Accordingly, absolute packet-loss rates, latency characteristics, and reconnection times are not directly generalizable to other environments. Furthermore, derived indicators (e.g., inferred missing samples and UI-side effective update rates) provide practical observability but are not a substitute for synchronized,

end-to-end timing instrumentation across all components. These limitations are typical for applied integration work and motivate careful interpretation of results when the system is transferred to different hardware, radio environments, or production-like conditions [AKI18, KLE17]. Despite these constraints, the internal consistency between UI indicators, stored time-series data, and AAS-exposed machine signals supports the validity of the demonstrated integration approach [MON21, BAS25, PLA23].

Implications for future work. The implemented system provides a reliable foundation for subsequent extensions, such as richer semantic modeling of external sensors, integration of additional machine signals, or higher-level analytics (e.g., condition monitoring and predictive maintenance) as described in the literature [JAR06, WANG21, SIEM18, TAO19]. Importantly, the architecture already provides the core prerequisites for such extensions: dependable ingestion, traceable persistence, and a standardized semantic publication channel for machine state [AAS20, BAS25, MON21].

8 Conclusion and Outlook

8.1 Conclusion

This thesis addressed the challenge of dependable data acquisition, integration, and semantic publication in an industrial machining context characterized by heterogeneous interfaces and distributed data sources. The primary objective was to design and implement a monitoring and integration architecture that enables reliable access to machine-internal signals and external sensor data, supports live visualization and persistent storage, and provides a standards-based semantic representation of selected machine states.

The implemented system integrates machine-internal signals from an OPC UA enabled CNC machine with external sensor streams acquired via serial and TCP-based interfaces [OPC24, MAH09]. A modular backend architecture delivers live telemetry to a web-based user interface using WebSocket communication [RFC11], while high-frequency sensor data are persistently stored using MongoDB time-series collections [MON21]. The evaluation results show that, for the tested nominal, stress, and fault-injection scenarios, the pipeline remained operationally usable, recovered automatically from transient failures, and provided runtime observability (e.g., connection state and loss indicators) sufficient to interpret data continuity and quality [AKI18, KLE17].

A key contribution of this work is the selective semantic publication of machine-internal signals using the Asset Administration Shell (AAS). By mapping relevant OPC UA nodes into AAS submodels hosted via Eclipse BaSyx, the system enables standardized access to machine state information without coupling semantic modeling directly to low-level data acquisition [AAS20, PLA23, BAS25]. The evaluation confirms that the published AAS elements are addressable through the intended `idShortPath` structure and remain consistent with the middleware snapshot values, supporting integration into a broader Digital Twin context [PLA23, TAO19].

Overall, the results demonstrate that dependable monitoring for the considered use case does not require fully lossless data streams or monolithic system designs. Instead, robustness, transparency, and recoverability were achieved through explicit handling of heterogeneous interfaces, modular service separation, and clear instrumentation of runtime behavior [AKI18, KLE17]. The implemented architecture fulfills the thesis objectives and provides a practical reference for data integration in applied industrial research environments.

8.2 Outlook

While the implemented system meets the defined goals, several directions for future work emerge naturally from the presented results.

A first extension concerns semantic modeling. External sensor data were intentionally excluded from formal AAS modeling in this thesis to limit complexity at the integration stage. Future

work could integrate selected sensor streams into standardized AAS submodels, for example by adopting condition monitoring or maintenance templates provided by the Industrial Digital Twin Association [IDT24, PLA23]. This would strengthen interoperability and enable cross-asset analysis within a Digital Twin ecosystem.

A second direction relates to scalability and deployment. The current implementation targets a single-machine laboratory setup. Extending the architecture to multiple machines and sensor instances would require coordinated naming conventions, scalable registry usage, and systematic management of AAS identifiers. Container orchestration and automated configuration generation could support such deployments while preserving the modular structure demonstrated in this thesis [MER14, BAS25].

From an analytics perspective, the persisted time-series data provide a suitable basis for higher-level processing. Future work could incorporate online or offline analytics for condition monitoring, anomaly detection, or predictive maintenance, building on established approaches described in the literature [JAR06, WANG21, SIEM18]. The current system already provides key prerequisites for such extensions, including traceable persistence, session metadata, and semantic access to machine state [MON21, AAS20].

Finally, more comprehensive evaluation under production-like conditions would strengthen external validity. Experiments across different network environments, sensor firmware versions, and longer operating periods would provide additional insight into long-term stability and performance. Nevertheless, the results presented in this thesis indicate that the chosen architectural principles are sound and transferable to comparable industrial monitoring scenarios [AKI18, KLE17].

In summary, this thesis establishes a dependable and extensible foundation for industrial data integration and monitoring. By combining pragmatic engineering decisions with standards-based semantic publication, it demonstrates how heterogeneous industrial data sources can be integrated into a coherent and operationally usable system [PLA23, TAO19].

9 References

AAS20 Plattform Industrie 4.0 and Industrial Digital Twin Association (IDTA): *Details of the Asset Administration Shell, Part 1: The Metamodel*, Version 3.0. 2020. Online: https://rb.gy/h0v6e0 [Accessed: 04 Nov 2020].

AKI18 T. Akidau, S. Chernyak, and R. Lax: *Streaming Systems: The What, Where, When, and How of Large-Scale Data Processing*. O'Reilly Media, 2018.

BAS25 Eclipse BaSyx (Fraunhofer IESE): *Eclipse BaSyx Documentation and Components*. 2023–2025. Online: https://www.eclipse.org/basyx/ [Accessed: 04 Nov 2025].

BHT25 Berlin University of Applied Sciences and Technology (BHT): *Official Website*. Online: https://www.bht-berlin.de/ [Accessed: 05 Nov 2025].

CAV20 S. Cavalieri and M. G. Salafia: *Asset Administration Shell for PLC Representation Based on IEC 61131–3*. IEEE Access, vol. 8, pp. 142606–142621, 2020. Online: https://doi.org/10.1109/ACCESS.2020.3013890 [Accessed: 19 Dec 2025].

CHA25 Chart.js Contributors: *Chart.js Documentation*. Online: https://www.chartjs.org/ [Accessed: 05 Nov 2025].

DAZA20 V. Daza, J. L. Hernández, and A. García: *Middleware Solutions for the Industrial Internet of Things: A Survey*. Sensors, vol. 20, no. 17, 2020. Online: https://doi.org/10.3390/s20174812 [Accessed: 19 Dec 2025].

EMO25 Fraunhofer Institute for Open Communication Systems (FOKUS): *EMOTION — Edge-Based Monitoring and Optimization for Industrial Machining*. Online: https://www.fokus.fraunhofer.de/de/dps/projekte/emotion.html [Accessed: 19 Dec 2025].

FAS25 S. Ramírez et al.: *FastAPI Documentation*. 2020–2025. Online: https://fastapi.tiangolo.com/ [Accessed: 01 Nov 2025].

GRI17 M. Grieves and J. Vickers: *Digital Twin: Mitigating Unpredictable, Undesirable Emergent Behavior in Complex Systems*. In: *Transdisciplinary Perspectives on Complex Systems*. Springer, 2017. Online: https://doi.org/10.1007/978-3-319-38756-7_4 [Accessed: 19 Dec 2025].

HONG18 S. Hong et al.: *Comparison of OPC UA and MQTT for Industrial Communication*. IEEE ICIT, 2018. Online: https://doi.org/10.1109/ICIT.2018.8352285 [Accessed: 19 Dec 2025].

IDT24 Industrial Digital Twin Association (IDTA): *AAS Submodel Templates (e.g., Nameplate, Condition Monitoring, Maintenance)*. 2022–2024. Online: https://industrialdigitaltwin.org/ [Accessed: 03 Nov 2025].

IPK25 Fraunhofer Institute for Production Systems and Design Technology (IPK): *Official Website and Research Profile*. Online: https://www.ipk.fraunhofer.de/ [Accessed: 05 Nov 2025].

JAR06 A. K. S. Jardine, D. Lin, and D. Banjevic: *A Review on Machinery Diagnostics and Prognostics Implementing Condition-Based Maintenance*. Mechanical Systems and Signal Processing, vol. 20, no. 7, pp. 1483–1510, 2006. Online: https://doi.org/10.1016/j.ymssp.2005.09.012 [Accessed: 19 Dec 2025].

JEN17 S. K. Jensen, T. B. Pedersen, and C. Thomsen: *Time Series Management Systems: A Survey*. IEEE Transactions on Knowledge and Data Engineering, vol. 29, no. 11, pp. 2581–2600, 2017. Online: https://doi.org/10.1109/TKDE.2017.2740932 [Accessed: 19 Dec 2025].

JSO25 JSONata: *JSONata Language Reference*. Online: https://jsonata.org/ [Accessed: 02 Nov 2025].

KLE17 M. Kleppmann: *Designing Data-Intensive Applications*. O'Reilly Media, 2017.

KRA23 P. Kraus, J. Schmid, and S. Reinhart: *Digital Twins in Manufacturing: A Systematic Literature Review*. Computers in Industry, vol. 145, 2023. Online: https://doi.org/10.1016/j.compind.2022.103812 [Accessed: 19 Dec 2025].

LEE14 J. Lee, B. Bagheri, and H.-A. Kao: *A Cyber-Physical Systems Architecture for Industry 4.0-Based Manufacturing Systems*. Manufacturing Letters, vol. 3, pp. 18–23, 2015. Online: https://doi.org/10.1016/j.mfglet.2014.12.001 [Accessed: 19 Dec 2025].

MAH09 W. Mahnke, S.-H. Leitner, and M. Damm: *OPC Unified Architecture*. Springer, 2009.

MER14 D. Merkel: *Docker: Lightweight Linux Containers for Consistent Development and Deployment*. Linux Journal, vol. 2014, no. 239, 2014.

MON21 MongoDB Inc.: *MongoDB Manual 5.0 — Time-Series Collections*. 2021–2024. Online: https://docs.mongodb.com/ [Accessed: 01 Nov 2025].

OPC24 OPC Foundation: *OPC Unified Architecture Specifications, Parts 1–21*. 2009–2024. Online: https://opcfoundation.org/ [Accessed: 04 Nov 2025].

PLA23 Plattform Industrie 4.0: *Details of the Asset Administration Shell, Parts 1–3*. 2020–2023. Online: https://www.plattform-i40.de/ [Accessed: 03 Nov 2025].

QUA25 Quart Project: *Quart Documentation*. Online: https://pgjones.gitlab.io/quart/ [Accessed: 03 Nov 2025].

REA25 Meta: *React Documentation*. Online: https://react.dev/ [Accessed: 02 Nov 2025].

RFC11 I. Fette and A. Melnikov: *The WebSocket Protocol*. RFC 6455, IETF, 2011. Online: https://datatracker.ietf.org/doc/html/rfc6455 [Accessed: 02 Nov 2025].

SCH25a SCHUNK GmbH & Co. KG: *iTENDO^2 — Assembly and Operating Instructions (easy connect / pad variants)*. Online: https://schunk.com/at/de/services/downloads/dokumente [Accessed: 12 Nov 2025].

SCH25b SCHUNK GmbH & Co. KG: *iTENDO^2 — Product Overview / Brochure*. Online: https://schunk.com/de/en/products/toolholding-technology/itendo2 and https://schunk.com/de/en/tool-clamping-technology/toolholders/tendo-hydraulic-expansion-chuck/itendo2/c/PGR_6468 [Accessed: 12 Nov 2025].

SIEM18 M. Siemens, A. García, and T. Schneider: *Predictive Maintenance in Industry 4.0: A Survey*. Procedia CIRP, vol. 72, pp. 998–1003, 2018. Online: https://doi.org/10.1016/j.procir.2018.03.292 [Accessed: 19 Dec 2025].

TAO19 F. Tao, A. Liu, T. Hu, and A. Y. C. Nee: *Digital Twin Driven Smart Manufacturing*. Academic Press, 2019.

VERM18 O. Vermesan et al.: *Internet of Things Beyond the Hype: Research, Innovation and Deployment*. River Publishers, 2018. Online: https://www.riverpublishers.com/research_details.php?book_id=714 [Accessed: 19 Dec 2025].

WANG21 J. Wang, Y. Ma, L. Zhang, R. X. Gao, and D. Wu: *A Comprehensive Review of Predictive Maintenance Approaches in Industry 4.0*. Journal of Manufacturing Systems, vol. 59, pp. 19–36, 2021. Online: https://doi.org/10.1016/j.jmsy.2021.03.004 [Accessed: 19 Dec 2025].

10 Appendix

Swarmy User Interface Visualization

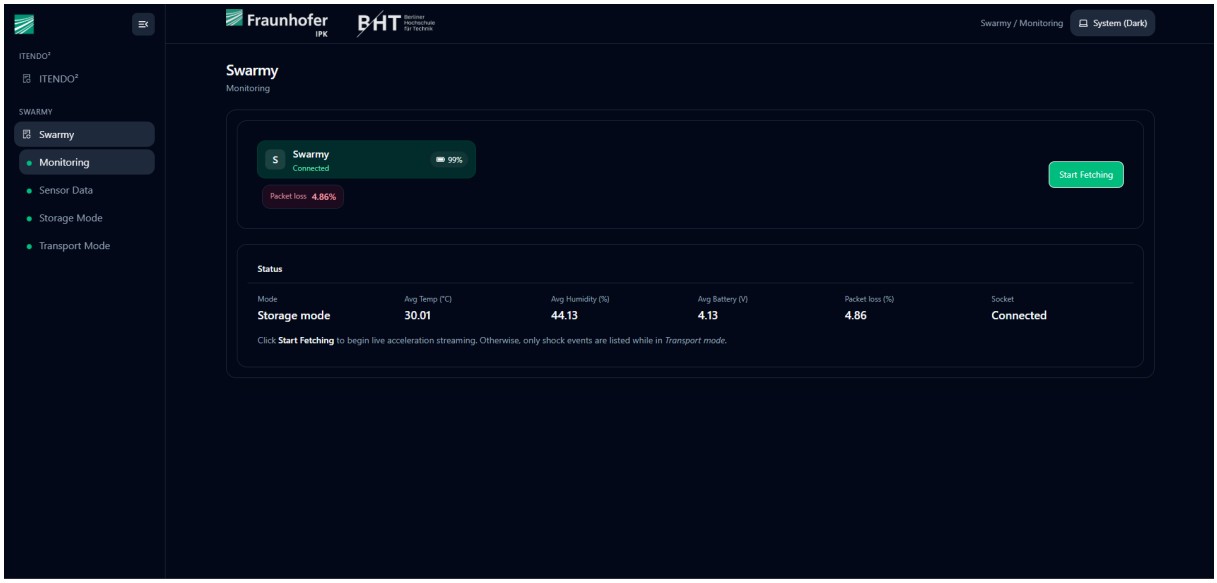


Fig. 3: Swarmy monitoring view with status, mode, battery, and packet-loss indicator.



Fig. 4: Swarmy monitoring view with live three-axis acceleration time-series during active measurement.



Fig. 5: Swarmy storage view with historical temperature, humidity, and battery voltage.

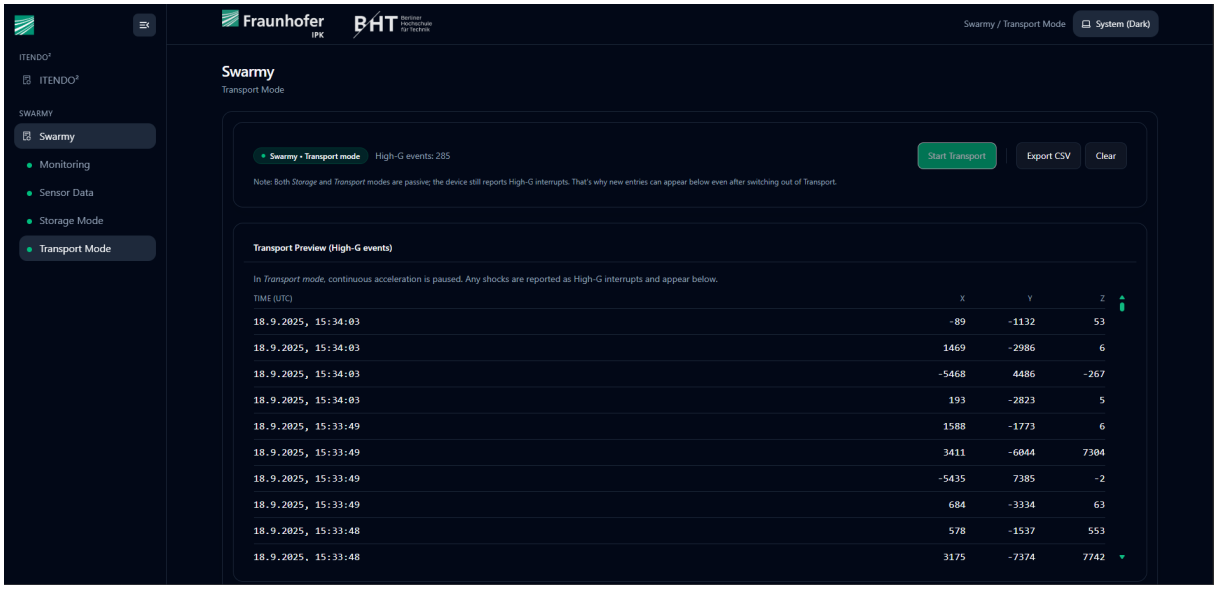


Fig. 6: Swarmy transport view showing recorded High-G impact events.

B. ITENDO^2 User Interface Visualization



Fig. 7: ITENDO^2 monitoring view showing live acceleration data from the table-mounted sensor, including connection status and runtime indicators.

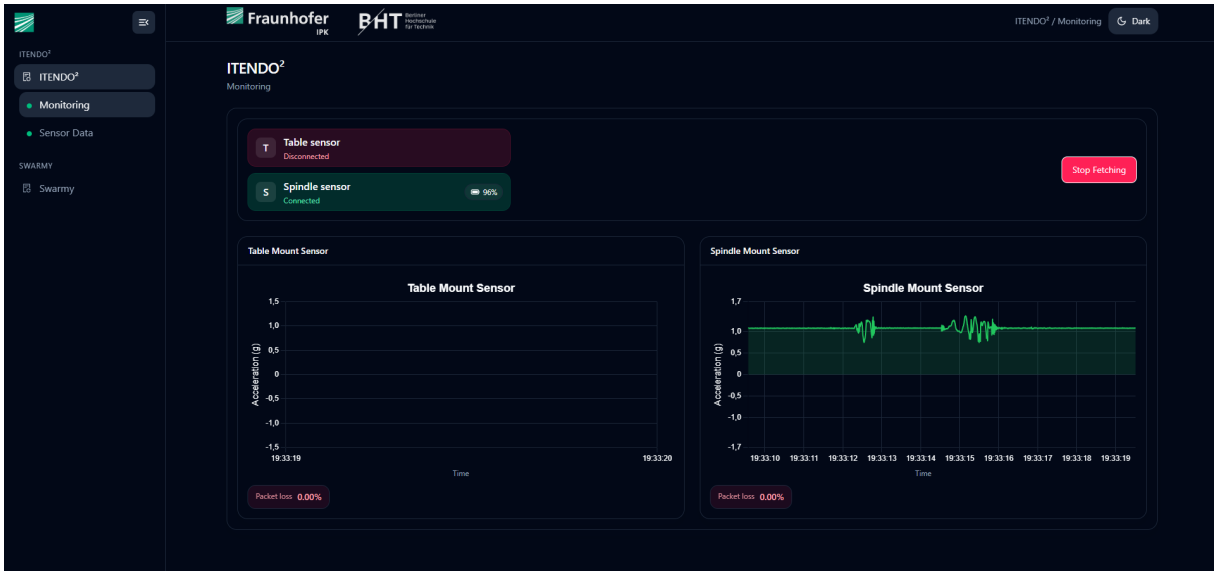


Fig. 8: ITENDO^2 monitoring view showing live acceleration data from the spindle-mounted sensor during active measurement.

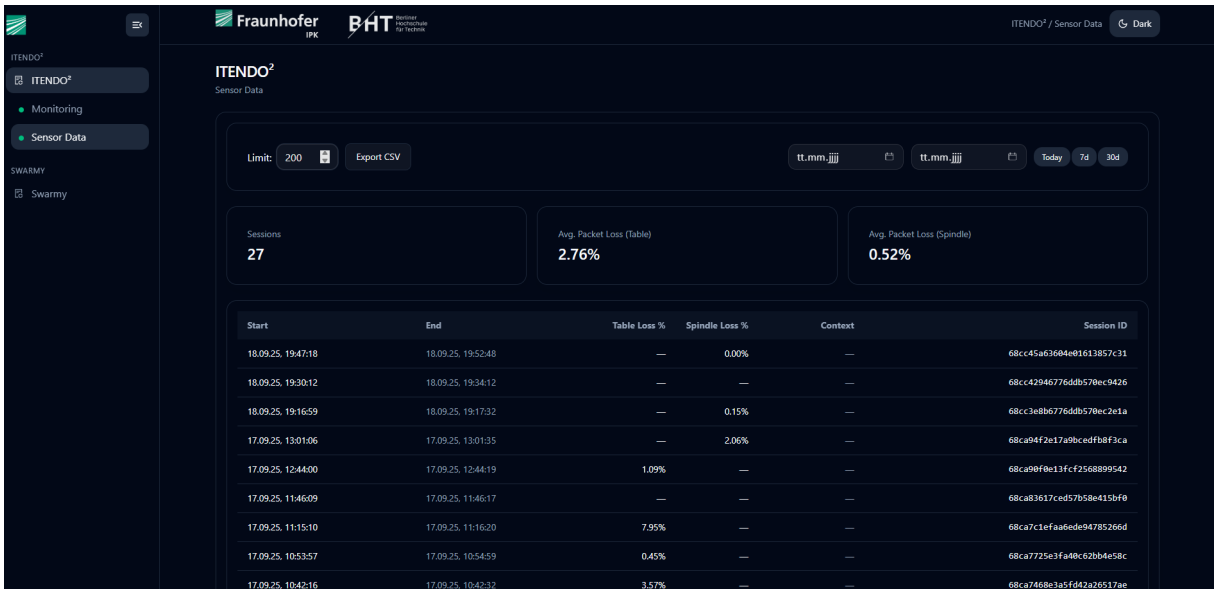


Fig. 9: ITENDO^2 session-based data view with recorded measurement sessions, average packet-loss KPIs, and CSV export functionality.

C. ITENDO^2 Control-Flow Diagram

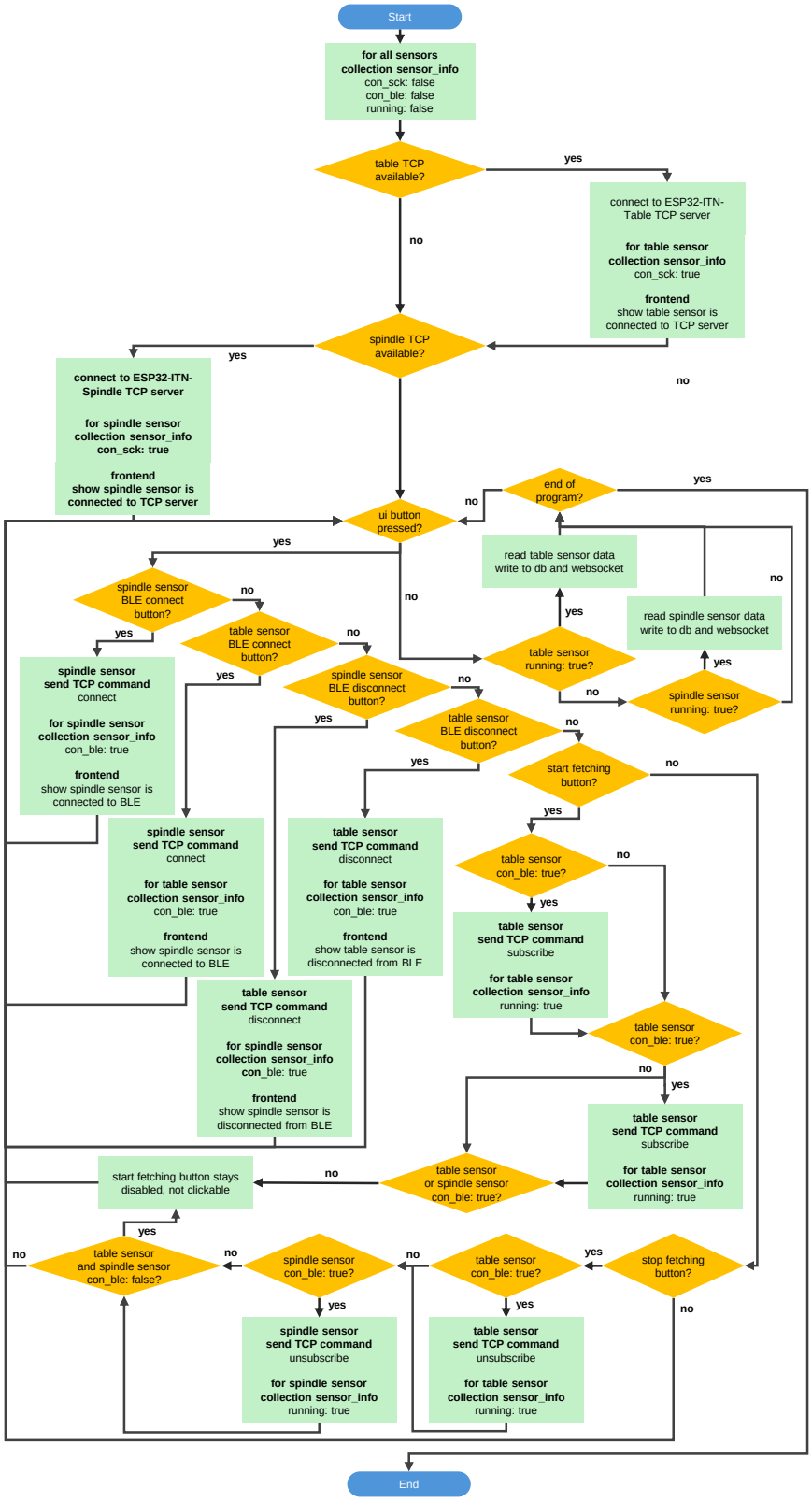


Fig. 10: Control-flow diagram for ITENDO^2 session handling: interaction between the user interface, backend services, ESP bridge nodes, and BLE-connected sensors.

D. OPC UA Middleware and DataBridge Configuration

```
{
  "timestamp": "2025-09-12T12:34:56.789Z",
  "values": {
    "ns=4;i=9": true,
    "ns=4;i=10": false
  },
  "valuesByName": {
    "Chuck2IsClosed": true,
    "Chuck2IsOccupied": false
  },
  "names": {
    "ns=4;i=9": "Chuck2IsClosed"
  }
}
```

Listing 5: Normalized response of the `GET /snapshot` endpoint: values and timestamps keyed by NodeId and friendly names.

```
NODES = {
  "Chuck2IsClosed": "ns=4;i=9",
  "Chuck2IsOccupied": "ns=4;i=10",
  "EmergencyStopExeron": "ns=4;i=3",
  "OperatingMode": "ns=4;i=2",
  "MaxArrayLength": "ns=0;i=11702",
  "SessionTimeoutCount": "ns=0;i=2281"
}
REVERSE = {v: k for k, v in NODES.items()}
```

Listing 6: Excerpt of the OPC UA NodeId mapping used by the middleware.

```
# middleware/opcua_manager.py (excerpt)
class OPCUAClientManager:
    def read_nodes(self, nodeids):
        self.ensure_connected()
        out = {}
        for nid in nodeids:
            try:
                node = self._client.get_node(nid)
                dv = node.get_data_value()
                out[nid] = {
                    "value": getattr(dv.Value, "Value", dv.Value),
                    "src_ts": dv.SourceTimestamp and dv.SourceTimestamp.isoformat(),
                    "srv_ts": dv.ServerTimestamp and dv.ServerTimestamp.isoformat(),
                    "error": None
                }
            except uaerrors.BadSessionClosed:
                self._connected = False
                self.connect()
```

```
                node = self._client.get_node(nid)
                dv = node.get_data_value()
                out[nid] = {
                    "value": getattr(dv.Value, "Value", dv.Value),
                    "src_ts": dv.SourceTimestamp and dv.SourceTimestamp.isoformat(),
                    "srv_ts": dv.ServerTimestamp and dv.ServerTimestamp.isoformat(),
                    "error": None
                }
            except Exception as e:
                out[nid] = {
                    "value": None,
                    "src_ts": None,
                    "srv_ts": None,
                    "error": str(e)
                }
        return out
```

Listing 7: Robust OPC UA read with transparent reconnect on `BadSessionClosed` (client manager excerpt): single-retry logic after session re-creation.

```
{
  "uniqueId": "source://exeron/Chuck2IsClosed",
  "serverUrl": "http://host.docker.internal:8090/snapshot"
}
```

Listing 8: DataBridge HTTP consumer: middleware `/snapshot` endpoint configuration for a selected signal.

```
$string($.values."ns=4;i=9")
```

Listing 9: JSONata extractor for `Chuck2IsClosed`: boolean value selection from the `values` object by NodeId.

```
{
  "uniqueId": "sink://exeron/Chuck2IsClosed",
  "submodelEndpoint": "http://<AAS-SERVER>/submodels/<id-encoded>",
  "idShortPath": "Chuck2IsClosed.Value",
  "api": "DotAAS-V3"
}
```

Listing 10: AAS sink configuration (DotAAS V3): transformed value write to the specified `idShortPath`.

```
[{"uniqueId": "timer1", "fixedRate": true,"delay": 0,"period": 1000}]
```

Listing 11: DataBridge timer definition: 1000 ms fixed-rate trigger for route execution.

```
{
  "datasource": "source://exeron/Chuck2IsClosed",
  "transformers": ["jsonataExtractChuck2IsClosed"],
  "datasinks": ["sink://exeron/Chuck2IsClosed"],
```

```
  "trigger": "timer",
  "triggerData": {"timerName": "timer1" }
}
```

Listing 12: DataBridge route example: HTTP consumer–JSONata–AAS sink connection bound to a timer trigger.

```
services:
  middleware:
    build: { context: ., dockerfile: Dockerfile }
    ports: [ "8090:8090" ]
    volumes: [ "./certs:/app/certs:ro" ]
    environment:
      PYTHONUNBUFFERED: "1"
    extra_hosts: [ "host.docker.internal:host-gateway" ]
  databridge:
    image: eclipsebasyx/databridge:1.0.0-milestone-08
    volumes: [ "./basyx_exeron/databridgeREST:/usr/share/config/" ]
    extra_hosts: [ "host.docker.internal:host-gateway" ]
```

Listing 13: Docker Compose configuration (excerpt) for middleware and BaSyx DataBridge: port and volume mappings with host resolution for local development.

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt /app/
RUN pip install --no-cache-dir -r requirements.txt
COPY middleware_software /app/middleware_software
EXPOSE 8090
CMD ["uvicorn","middleware_software.app.main:app","--host","0.0.0.0","--port","8090"]
```

Listing 14: Dockerfile for the FastAPI middleware (Python 3.11-slim): dependency installation, source copying, and Uvicorn application exposure.

E. AAS Visualization in Eclipse BaSyx

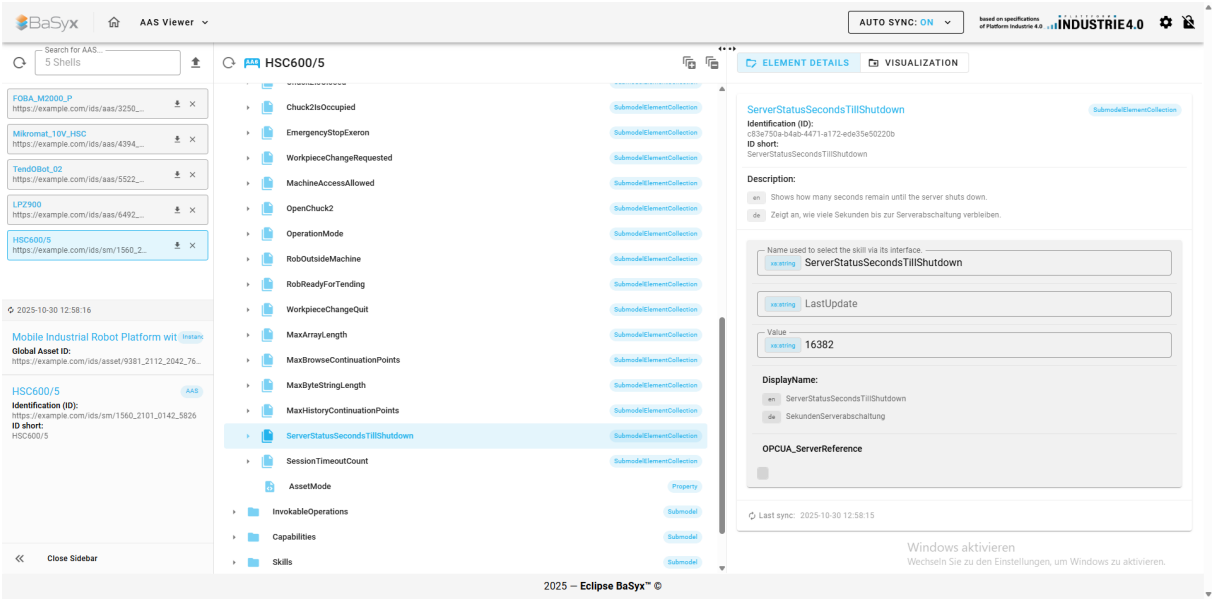


Fig. 11: ServerStatusSecondsTillShutdown property: remaining seconds before an automatic server shutdown.



Fig. 12: OpenChuck2 command/state: open or closed status of chuck 2 as exposed via OPC UA and mapped into the AAS.



Fig. 13: Chuck2IsClosed property: whether chuck 2 is currently closed, mapped from OPC UA to the AAS submodel.



Fig. 14: Chuck2IsOccupied property: whether chuck 2 is currently occupied by a workpiece in automatic operation.

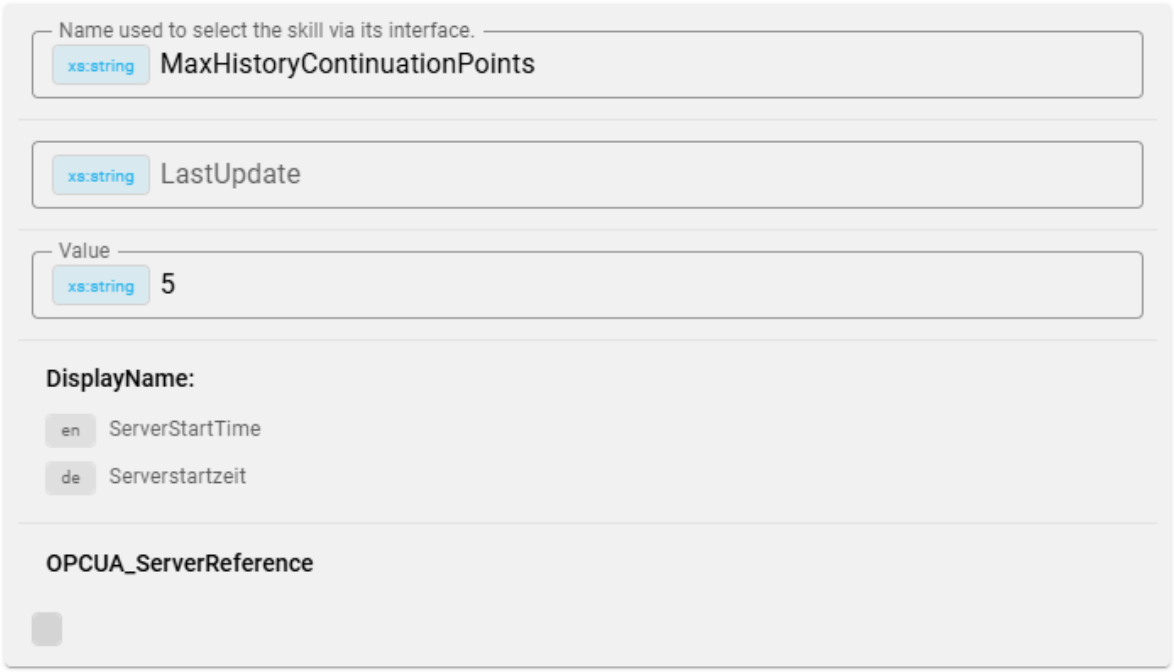


Fig. 15: MaxHistoryContinuationPoints property: maximum number of continuation points maintained by the OPC UA server for historical reads.

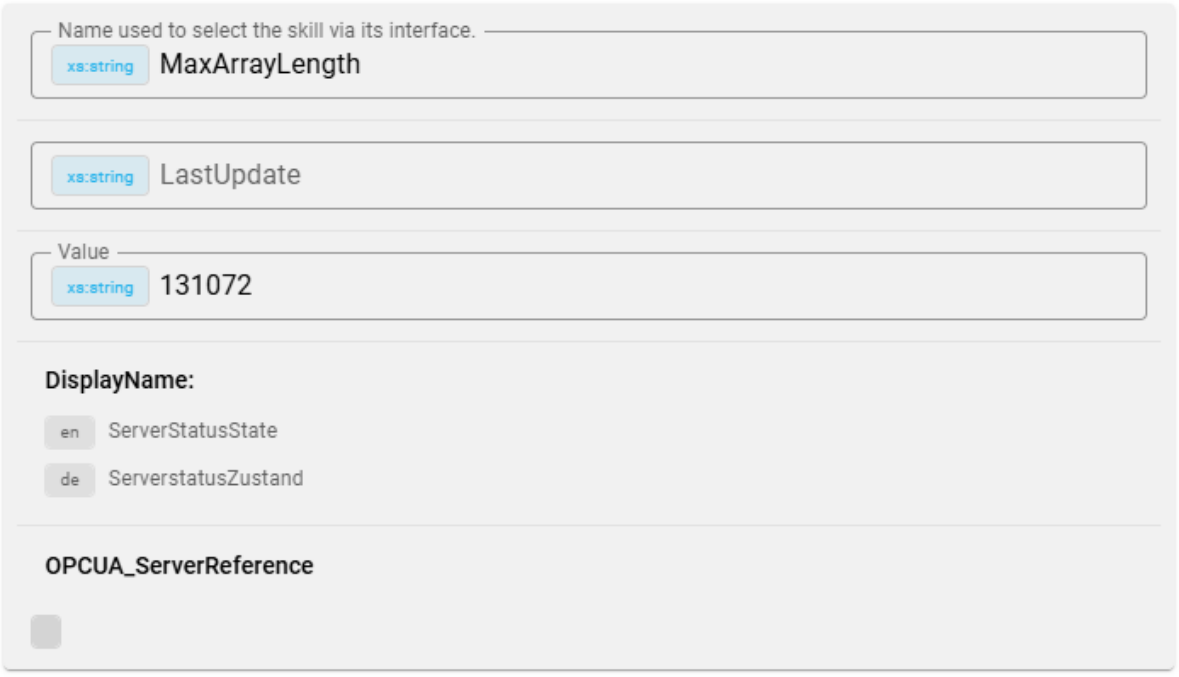


Fig. 16: MaxArrayLength property: maximum allowed length for array-type values in the OPC UA server.



Fig. 17: SessionTimeoutCount property: number of sessions that have reached their configured timeout.

11 Acknowledgments

I thank the Fraunhofer IPK EMOTION team for previous groundwork and helpful discussions, and Prof. Dr.-Ing. Francisco Morales Serrano for academic guidance. I would also like to thank my husband for his continuous support, encouragement, and understanding throughout the course of this work.